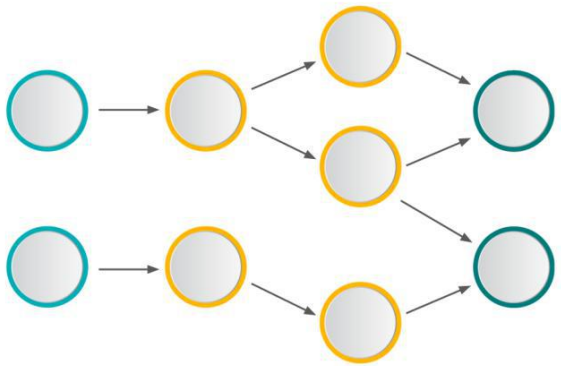
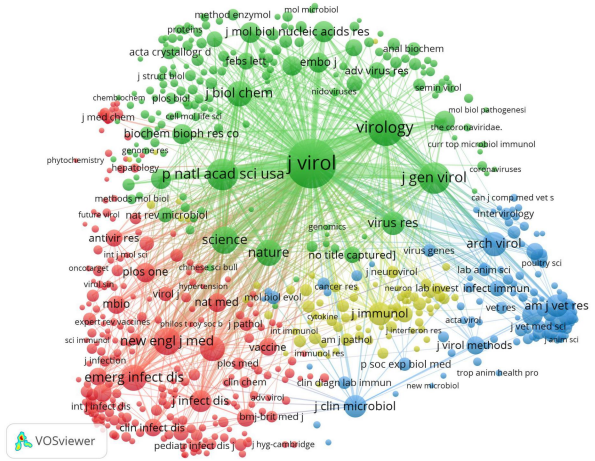
流计算与图计算

BIG DATA







流计算与图计算

CONTENTS

01 流计算
Streaming computation

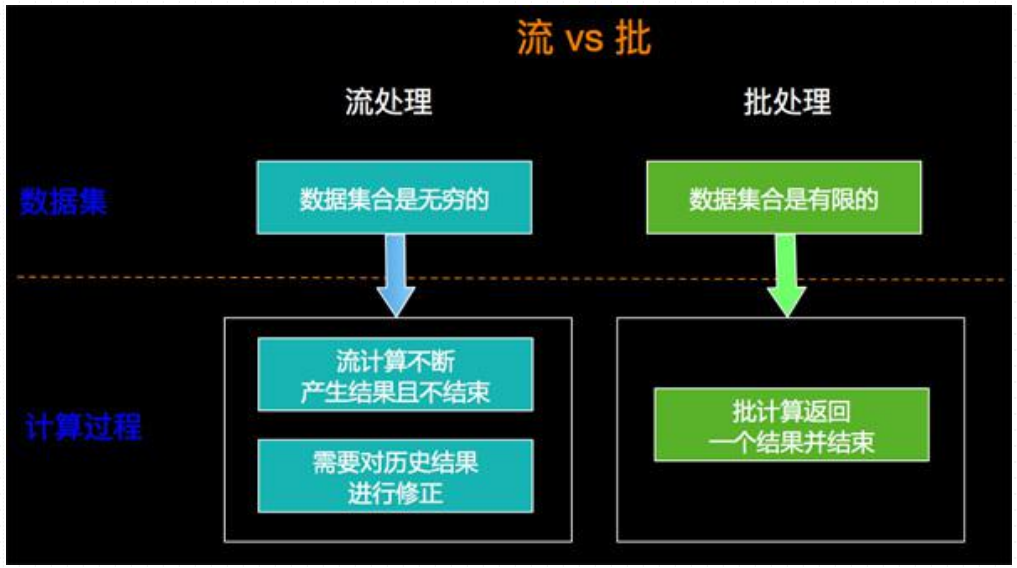
02 图计算
Graph computation

1.1 流计算概述

- 传统数据处理流程中总是先收集数据，然后将数据放到DB中。当需要候通过DB对数据做query相关处理。在一些实时搜索应用环境中，类似MapReduce方式的离线处理并不能很好地解决问题。这就引出了一种新的数据计算结构---流计算方式



批计算/流计算流程

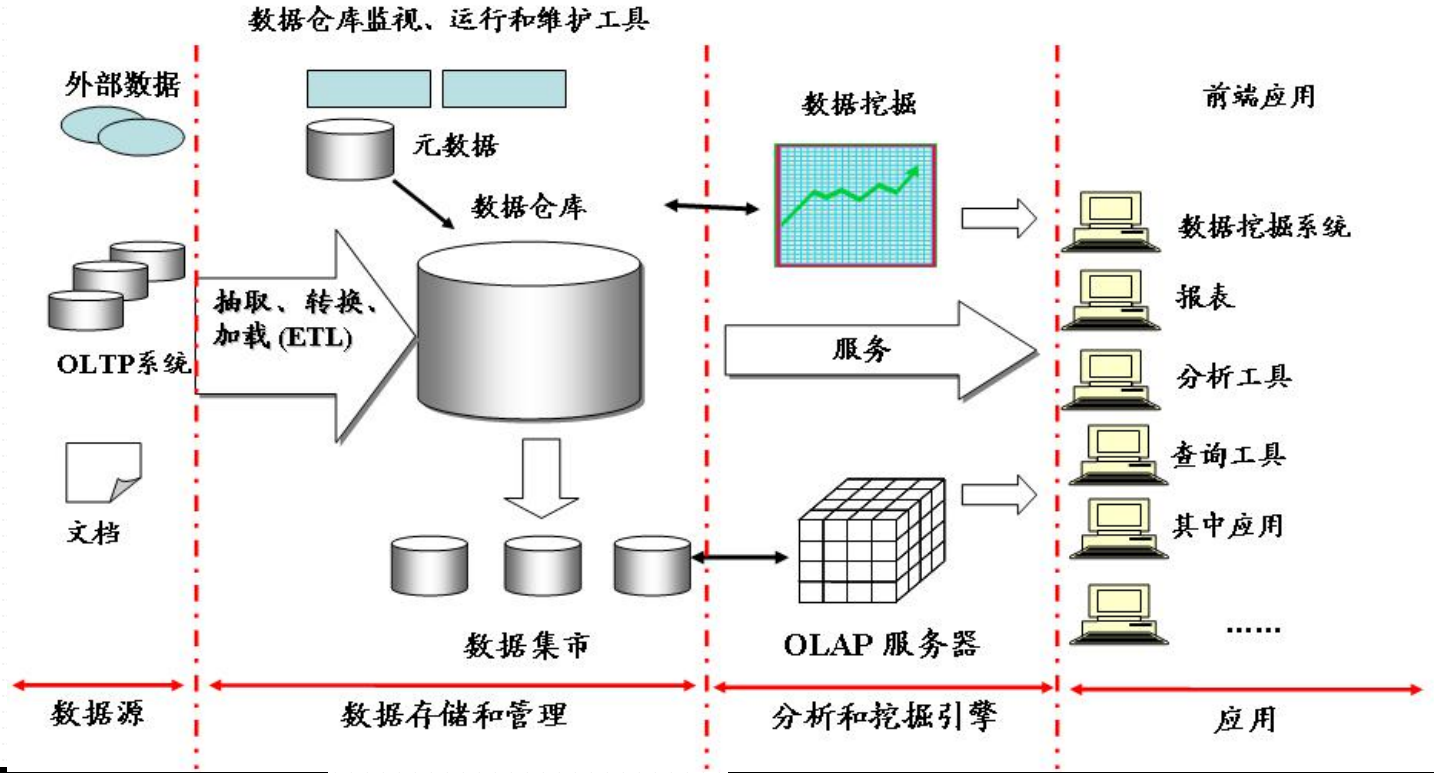


流计算与批计算对比

1.1.1 静态数据和流数据

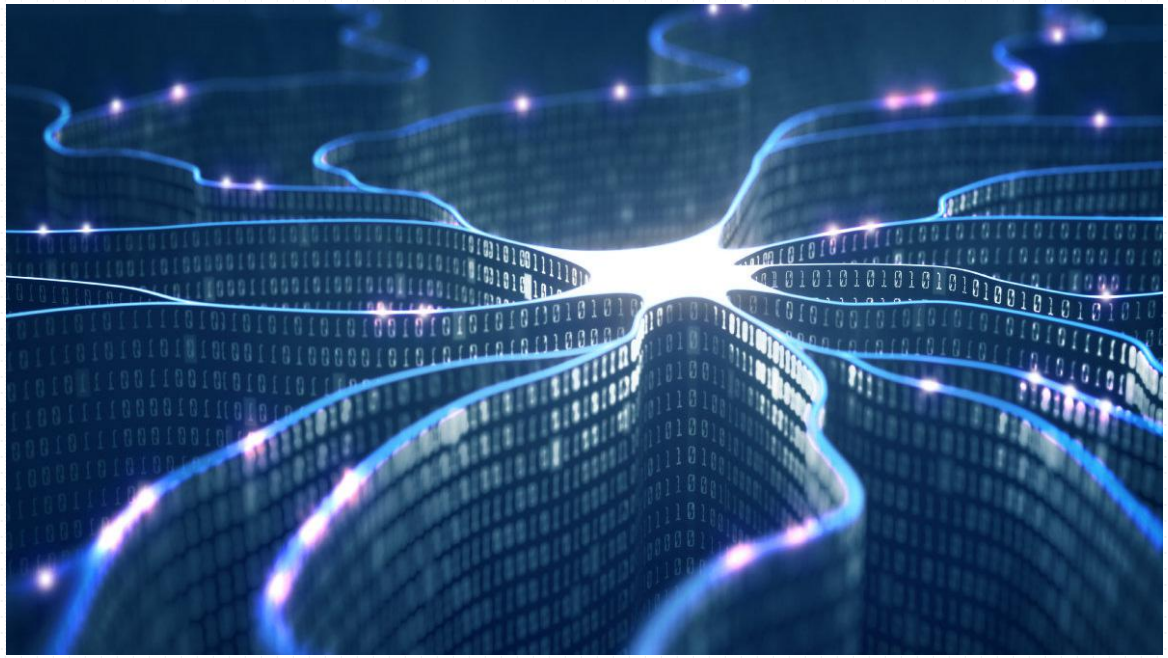
- 很多企业为了支持决策分析而构建的数据仓库系统，其中存放的大量历史数据就是静态数据。技术人员可以利用数据挖掘和OLAP（On-Line Analytical Processing）分析工具从静态数据中找到有价值的信息。

静态数据



1.1.1 静态数据和流数据

- 近年来，在Web应用、网络监控、传感监测等领域，兴起了一种新的数据密集型应用——流数据，即数据以大量、快速、时变的流形式持续到达
- 如：PM2.5检测、电子商务网站用户点击流

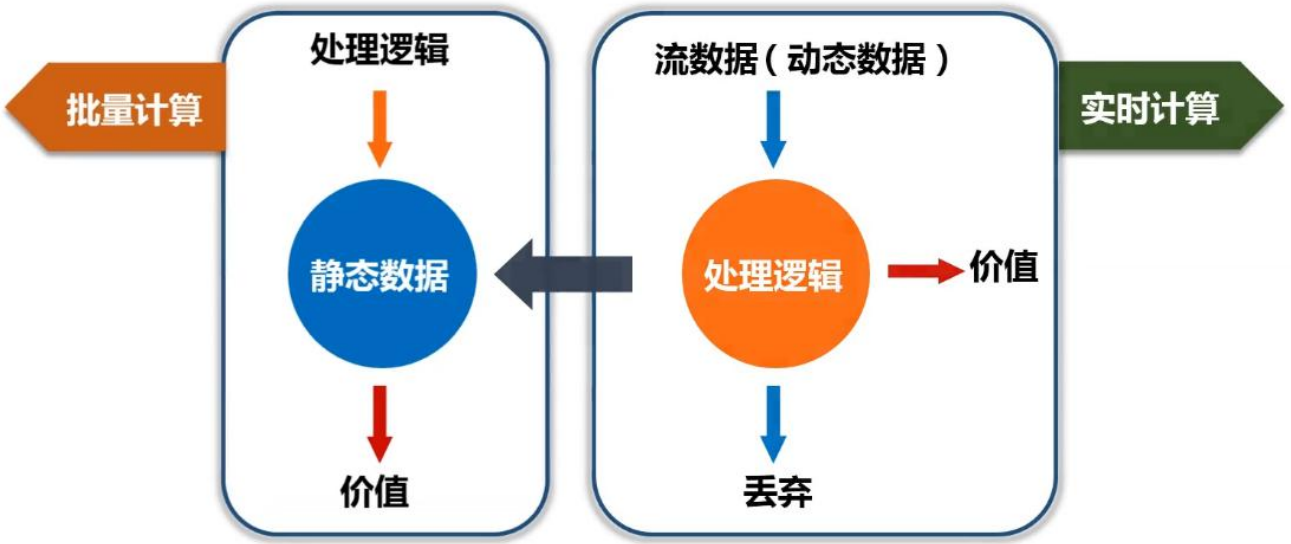


流 数 据 特 征

- 数据快速持续到达，也许无穷无尽
- 数据量大，但不十分关注存储，一旦经过处理，要么被丢弃，要么被归档存储
- 注重数据整体价值，不过分关注个别数据
- 数据顺序颠倒，或者不完整，系统无法控制将要处理的新到达的数据元素的顺序
- ….

1.1.2 批量计算和实时计算

- 对静态数据和流数据的处理，对应着两种截然不同的计算模式：**批量计算和实时计算**



- **批量计算**：充裕时间处理静态数据如Hadoop
- **实时计算**：流数据不适合采用批量计算，因为流数据不适合用传统的关系模型建模。流数据必须采用实时计算，响应时间为秒级

在大数据时代，数据格式复杂、来源众多、数据量巨大，对实时计算提出了很大的挑战。
针对流数据的实时计算——流计算，应运而生。

1.1.3 流计算概念

流计算：实时获取来自不同数据源的海量数据，经过实时分析处理获得有价值信息



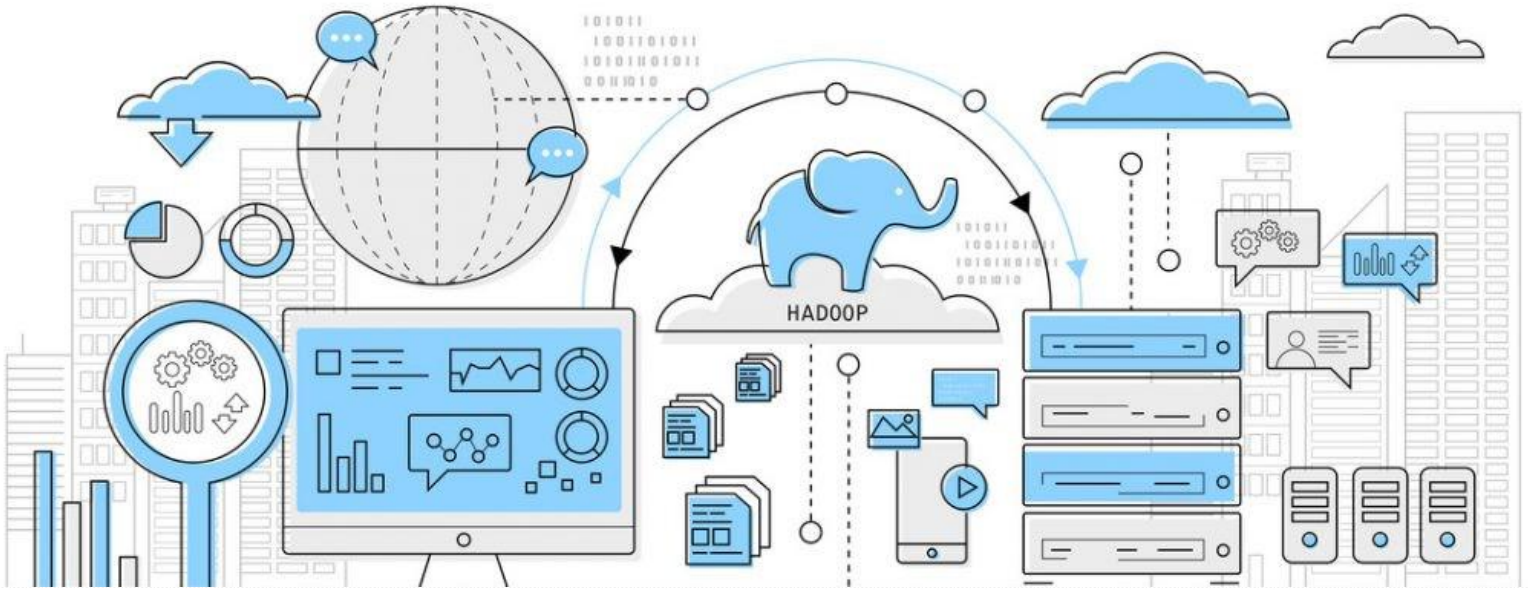
1.1.3 流计算概念

- 流计算秉承一个基本理念，即**数据的价值随着时间的流逝而降低，**如用户点击流。因此，当事件出现时就应该立即进行处理，而不是缓存起来进行批量处理。为了及时处理流数据，就需要一个低延迟、可扩展、高可靠的处理引擎
- 对于一个流计算系统来说，它应达到如下需求
  - **高性能**：处理大数据的基本要求，如每秒处理几十万条数据
  - **海量式**：支持TB级甚至是PB级的数据规模
  - **实时性**：保证较低的延迟时间，达到秒级别，甚至是毫秒级别
  - **分布式**：支持大数据的基本架构，必须能够平滑扩展
  - **易用性**：能够快速进行开发和部署
  - **可靠性**：能可靠地处理流数据

1.1.4 流计算与Hadoop

- Hadoop设计的初衷是面向大规模数据的批量处理，并行运行MapReduce任务，最后对结果进行汇总输出

- MapReduce是专门面向静态数据批量处理，内部机制为批处理做了高度优化，不适合处理持续的动态数据



1.1.5 流计算框架

- 三类常见的流计算框架和平台：商业级的流计算平台、开源流计算框架、公司为支持自身业务开发的流计算框架

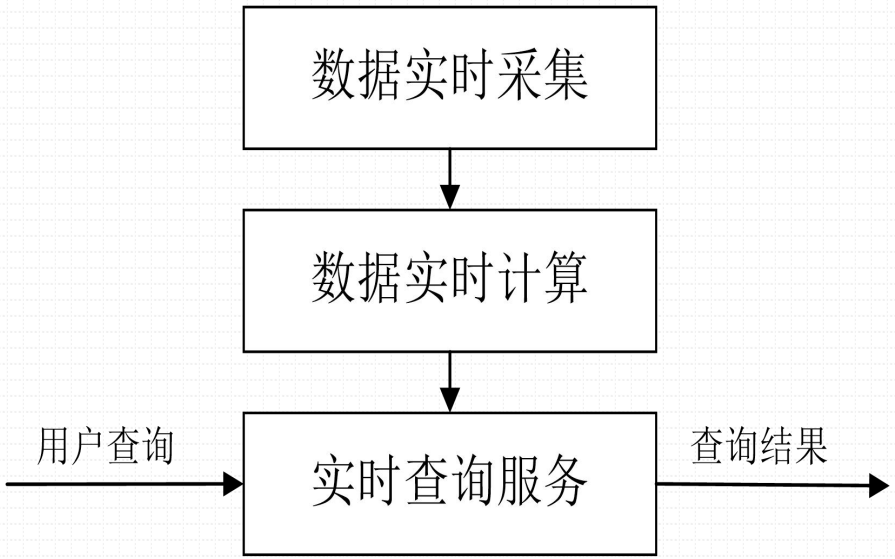
	Storm Record Acks	Spark Streaming, Trident Micro-Batching	Google Cloud Dataflow Transactional updates	Flink Distributed snapshots
数据保证	At least once	Exactly once	Exactly once	Exactly once
延迟	very low	High	low	very low
吞吐量	Low	High	Medium to High（由分布式的事务存储决定的）	High
计算模型	Streaming	Micro-batch	Streaming	Streaming
容错机制的开销	High	Low	由分布式事务存储决定	Low
流控	Problematic	Problematic	Natural	Natural
应用逻辑与容错的分离	Partially	NO（微批量的大小会影响语义）	Yes	Yes

常见流计算框架

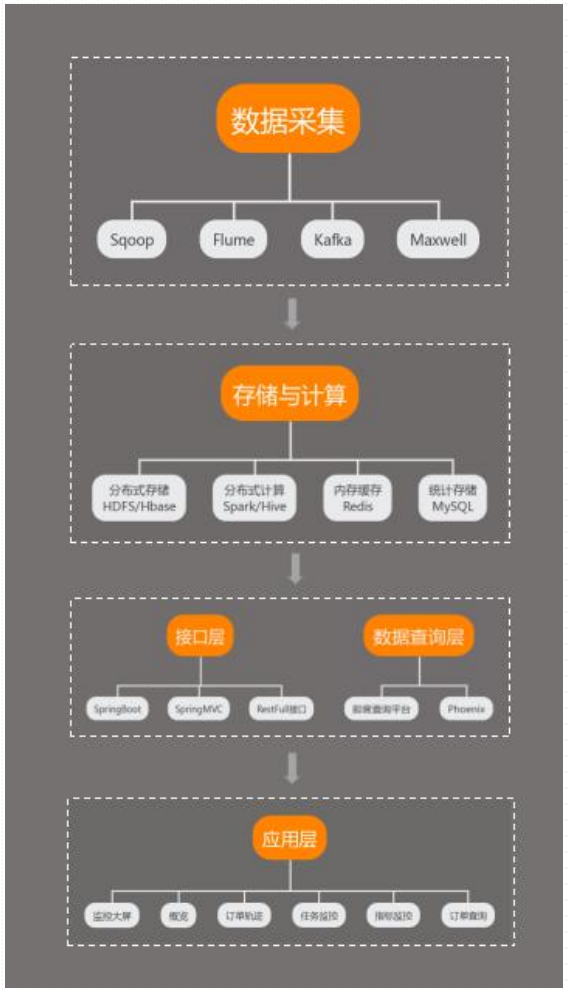
- **Apache Storm**：免费、开源的分布式实时计算系统，可简单、高效、可靠地处理大量的流数据
- **Spark Streaming**：Spark Streaming是Spark core API的扩展，支持实时数据流的处理，并且具有可扩展，高吞吐量，容错的特点。
- **Google Cloud Dataflow**：一种构建、管理和优化复杂数据处理流水线方法，集成许多内部技术。
- **Flink**：Apache Flink是面向数据流处理和批数据处理分布式开源计算框架，基于同一个Flink流式执行模型能够支持流处理和批处理两种应用类型。

1.2 流计算处理流程

- 流计算的处理流程一般包含三个阶段：数据实时采集、数据实时计算、实时查询服务（右图为滴滴数据流处理架构图）

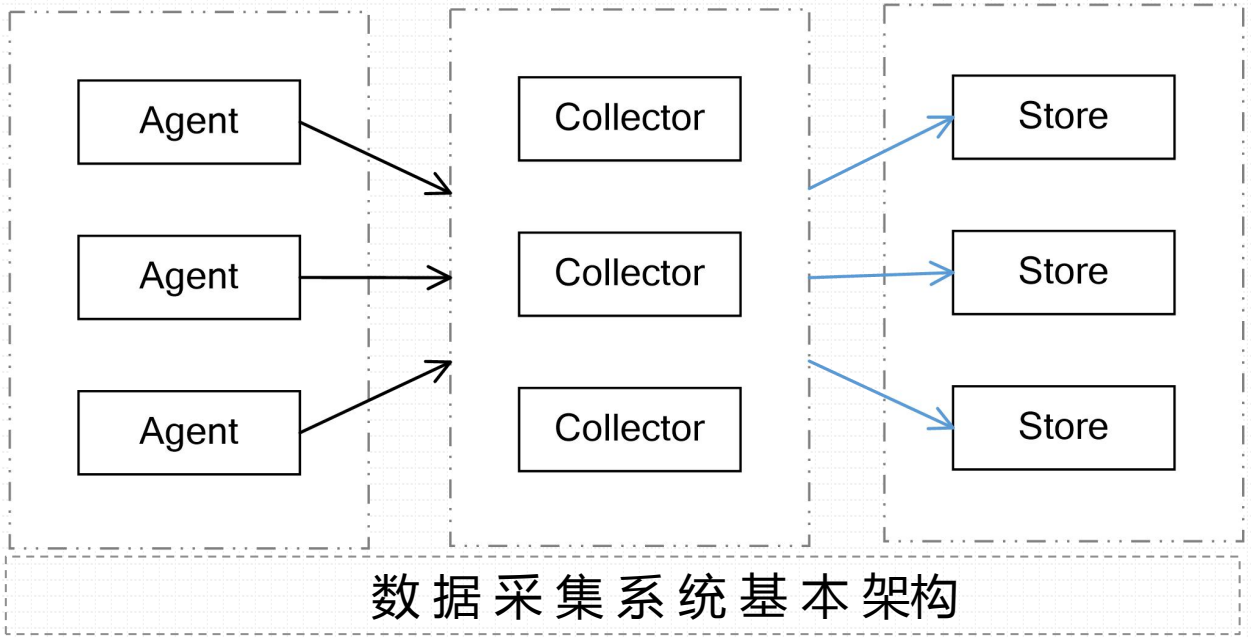


流计算处理流程示意图



1.2.1 数据实时采集

- 数据实时采集阶段通常采集多个数据源的海量数据，需保证实时性、低延迟与稳定可靠
- 目前有许多互联网公司发布的开源分布式日志采集系统均可满足每秒数百MB的数据采集和传输需求，如：
  - Facebook的Scribe
  - 淘宝的Time Tunnel
  - 基于Hadoop的Chukwa和Flume
- 采集系统基本架构
  - **Agent**：主动采集数据，并把数据推送到Collector部分
  - **Collector**：接收多个Agent的数据，并实现有序、可靠、高性能的转发
  - **Store**：存储Collector转发过来的数据（对于流计算不存储数据）



1.3 流计算应用

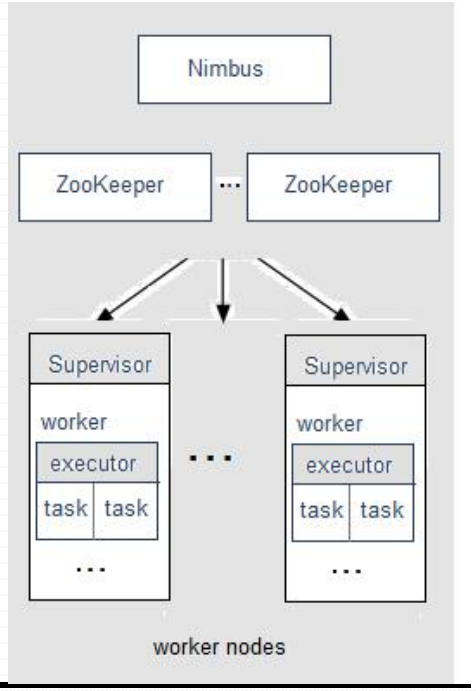
- 流计算是针对流数据的实时计算，可以应用在多种场景中，如Web服务、机器翻译、广告投放、自然语言处理、气候模拟预测等

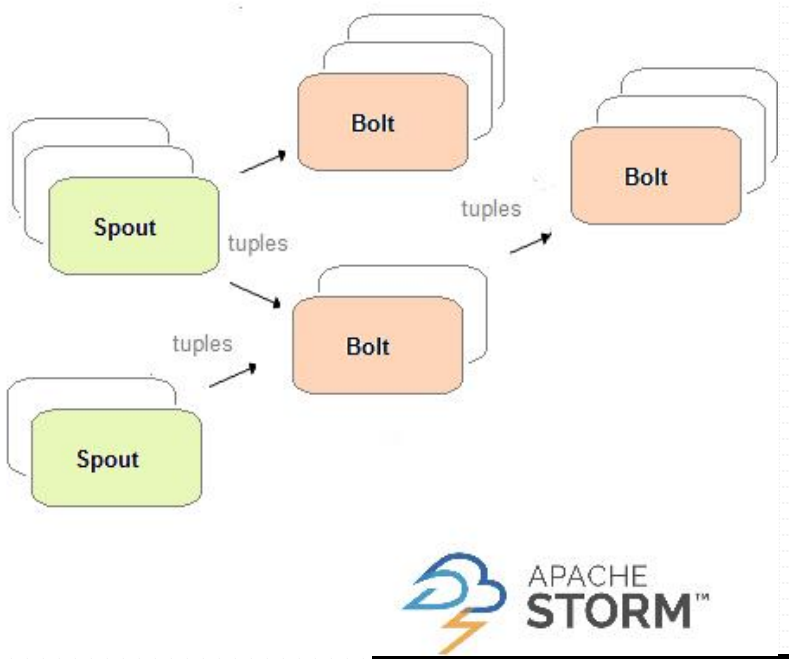
- 如百度、淘宝等大型网站中，每天都会产生大量流数据，包括用户的搜索内容、用户的浏览记录等数据。采用流计算进行实时数据分析，可以了解每个时刻的流量变化情况，甚至可以分析用户的实时浏览轨迹，从而进行实时个性化内容推荐

- 但是，并不是每个应用场景都需要用到流计算的。流计算适合于需要处理持续到达的流数据、对数据处理有较高实时性要求的场景

1.4 流计算开源框架 -- Storm

- Apache Storm是一个免费、开源的分布式实时计算系统，Storm对于实时计算的意义类似于Hadoop对于批处理的意义，Storm可以简单、高效、可靠地处理流数据，并支持多种编程语言
- Storm框架可以方便地与数据库系统进行整合，从而开发出强大的实时计算系统





1.4.1 Storm 特点

- Storm可用于许多领域中，如实时分析、在线机器学习、持续计算、远程RPC、数据提取加载转换等

- Storm具有以下主要特点：
  - 整合性：Storm可方便地与队列系统和数据库系统进行整合
  - 简易的API：Storm的API在使用上即简单又方便
  - 可扩展性：Storm的并行特性使其可以运行在分布式集群中
  - 可靠的消息处理：Storm保证每个消息都能完整处理
  - 免费、开源：Storm是一款开源框架，可以免费使用
  - ....

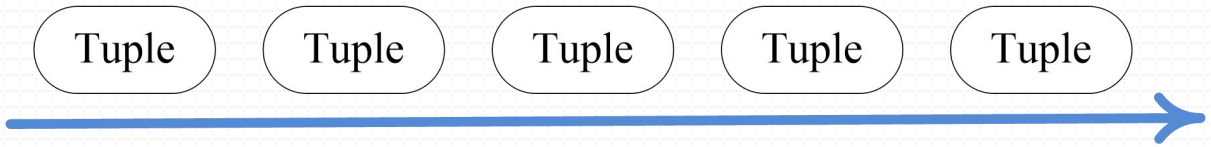
1.4.2 Storm 设计思想

Storm主要术语包括Streams、Spouts、Bolts、Topology和Stream Groupings

- **Streams**：Storm将流数据Stream描述成一个无限的Tuple序列，这些Tuple序列会以分布式的方式并行地创建和处理

Streams

无界的Tuple序列



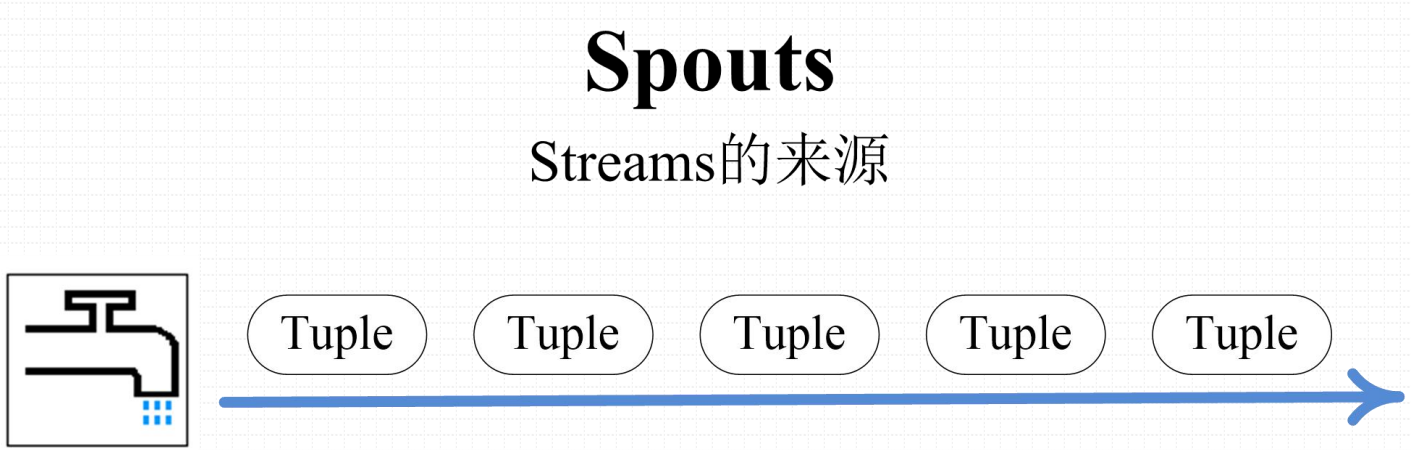
- 每个tuple是一堆值，每个值有一个名字，并且每个值可以是任何类型
- Tuple本来应该是一个Key-Value的Map，由于各个组件间传递的tuple的字段名称已经事先定义好了 ，所以Tuple只需要按序填入各个Value，所以就是一个Value List（值列表）

Field1	Field2	Field3	Field4

1.4.2 Storm 设计思想

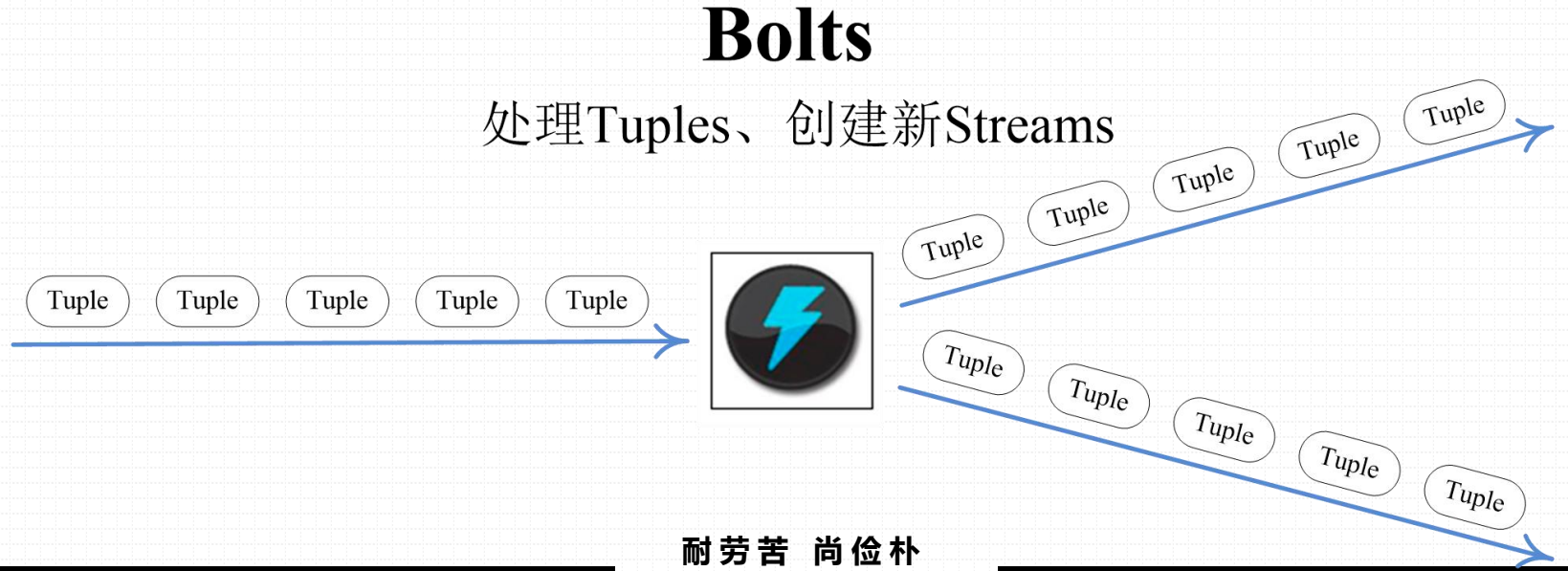
- **Spout：**Storm认为每个Stream都有一个源头，并把这个源头抽象为Spout

- 通常Spout会从外部数据源（队列、数据库等）读取数据，然后封装成Tuple形式发送到Stream中。Spout是一个主动的角色，在接口内部有个nextTuple函数，Storm框架会不停的调用该函数



1.4.2 Storm 设计思想

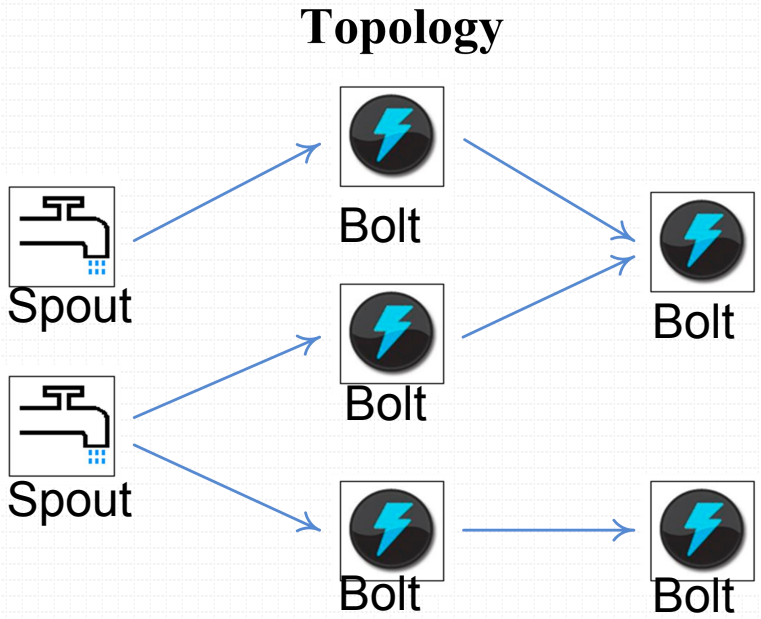
- **Bolt**：Storm将Streams的状态转换过程抽象为Bolt。Bolt即可以处理Tuple，也可以将处理后的Tuple作为新的Streams发送给其他Bolt
  - Bolt可以执行过滤、函数操作、Join、操作数据库等任何操作
  - Bolt是一个被动的角色，其接口中有一个execute(Tuple input)方法，在接收到消息之后会调用此函数，用户可以在此方法中执行自己的处理逻辑



1.4.2 Storm 设计思想

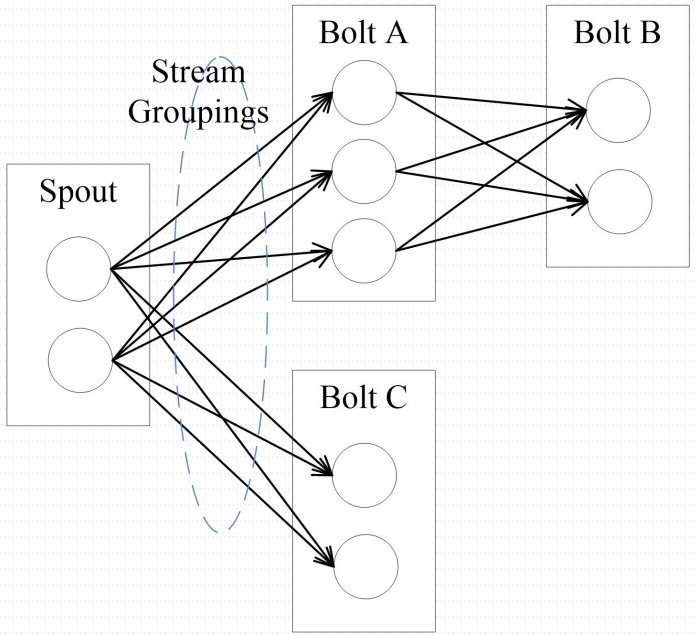
- **Topology**：Storm将Spouts和Bolts组成的网络抽象成Topology，它可被提交到Storm集群执行。Topology可视为流转换图，图中节点是一个Spout或Bolt，边则表示Bolt订阅了哪个Stream。当Spout或者Bolt发送元组时，它会把元组发送到每个订阅了该Stream的Bolt上进行处理

  - Topology里面的每个处理组件（Spout或Bolt）都包含处理逻辑，而组件之间的连接则表示数据流动的方向
  - Topology里面的每一个组件都是并行运行的
  - 在Topology里面可以指定每个组件的并行度，Storm会在集群里面分配那么多的线程来同时计算
  - 在Topology的具体实现上，Storm中的Topology定义仅仅是一些Thrift结构体（二进制高性能的通信中间件），支持各种编程语言进行定义



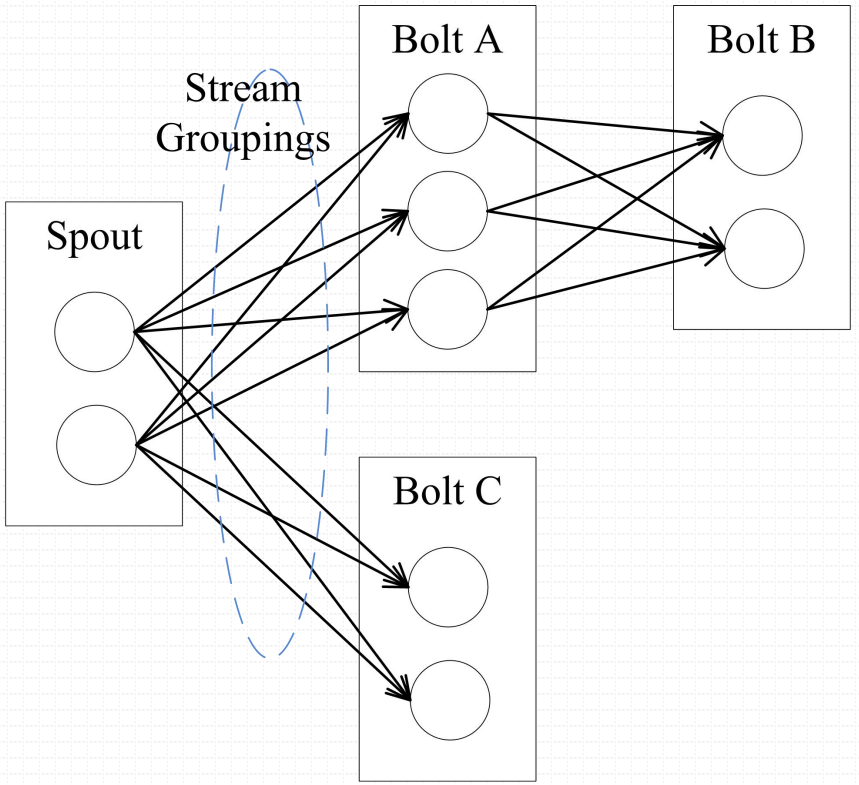
1.4.2 Storm 设计思想

- **Stream Groupings：**Storm中的Stream Groupings用于告知Topology如何在两个组件间（如Spout和Bolt之间，不同的Bolt之间）进行Tuple传送。每一个Spout和Bolt都可有多个分布式任务，一个任务在什么时候、什么方式发送Tuple由Stream Groupings决定



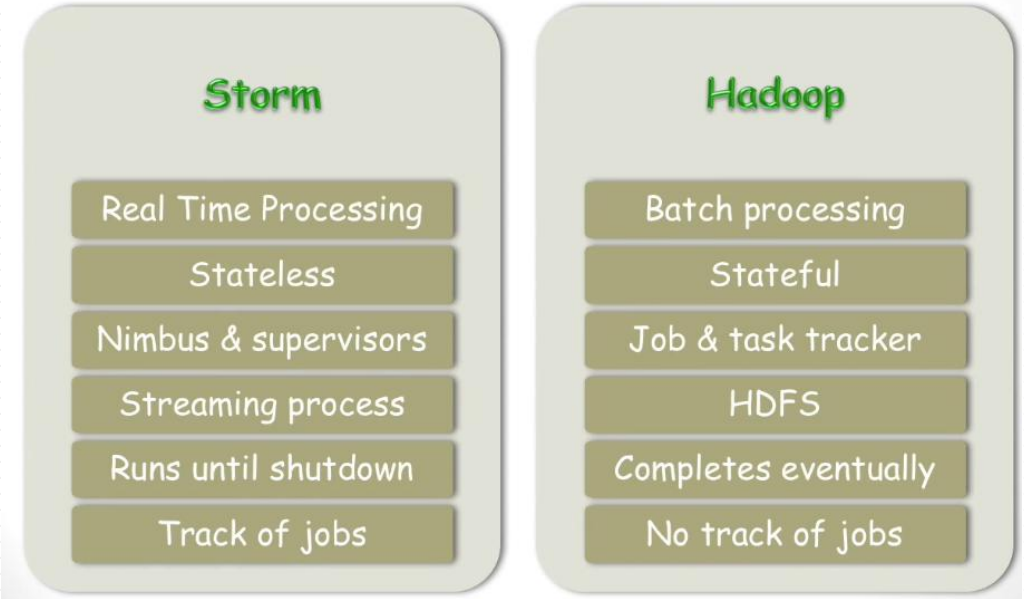
1.4.2 Storm 设计思想

- Storm中的Stream Groupings有如下几种方式：
  - **ShuffleGrouping**：随机分组分发Stream中的Tuple，保证每个Bolt的Task接收Tuple数量大致一致
  - **FieldsGrouping**：按照字段分组，保证相同字段的Tuple分配到同一个Task中
  - **AllGrouping**：广播发送，每一个Task收到所的Tuple
  - **GlobalGrouping**：全局分组所有Tuple发到同一个Task
  - **NonGrouping**：不分组，和ShuffleGrouping类似，当前Task的执行会和它的被订阅者在同一个线程中执行
  - **DirectGrouping**：直接分组由某Task来执行Tuple处理



1.4.3 Storm 框架设计

- Storm运行任务方式与Hadoop类似：Hadoop运行的是MapReduce作业，而Storm运行“Topology”
- 但两者的任务大不相同，主要在于：MapReduce作业最终会完成计算并结束运行，而Topology将持续处理消息（直到人为终止）

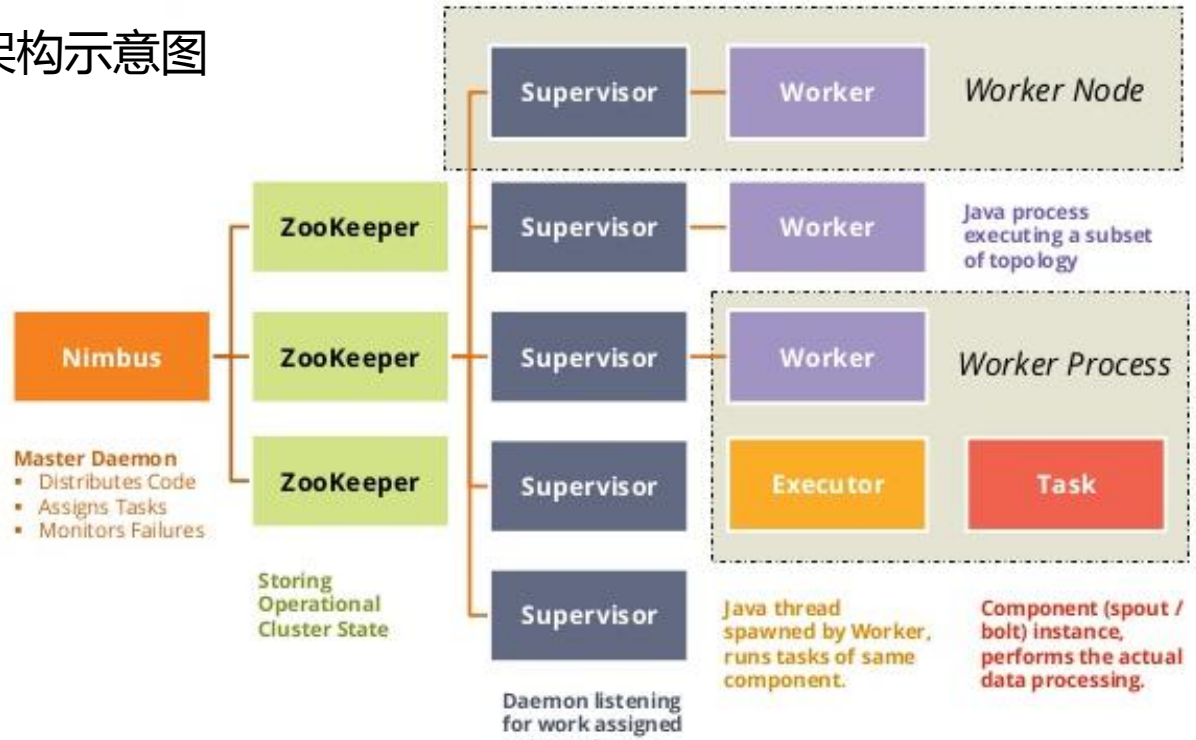


Storm和Hadoop架构组件功能对应关系

1.4.3 Storm 框架设计

- Storm使用Zookeeper来作为分布式协调组件，负责Nimbus和多个Supervisor之间的所有协调工作。借助于Zookeeper，若Nimbus进程或Supervisor进程意外终止，重启时也能读取、恢复之前的状态并继续工作，使得Storm极其稳定

Storm集群架构示意图

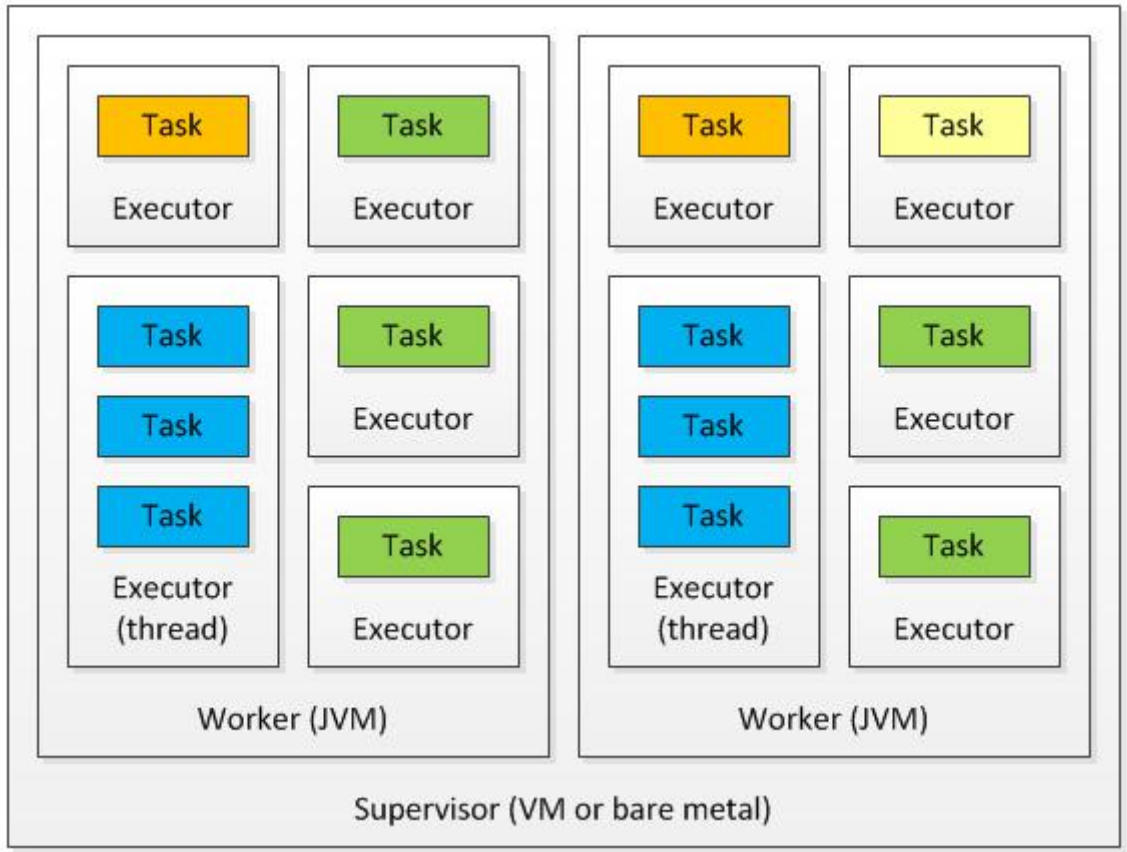


1.4.3 Storm 框架设计

- **worker**: 每个worker进程属于一个特定的Topology，每个Supervisor节点worker可有多个，每个worker对Topology中的每个组件（Spout或 Bolt）运行一个或者多个executor线程来提供task的运行服务

- **executor**：executor是产生于worker进程内部的线程，会执行同一个组件的一个或者多个task。

- **task**: 实际的数据处理由task完成，在Topology的生命周期中，每个组件的task数目是不会发生变化的，而executor的数目却不一定。executor数目小于等于task的数目，默认二者相等



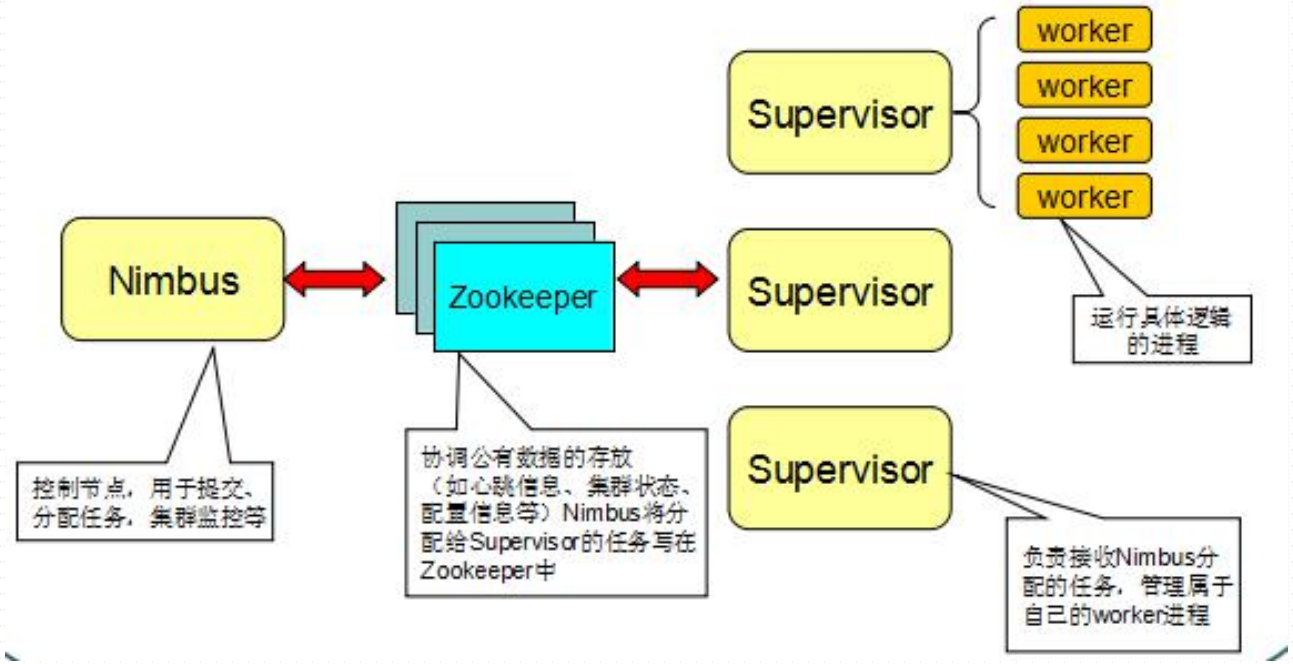
1.4.4 Storm 工作流程

- Storm工作流程

1. 所有Topology任务提交必须在Storm客户端节点上进行，由Nimbus节点分配其他Supervisor 节点进行处理

2. Nimbus节点首先将提交的Topology进行分片，分成一个个Task，分配给相应的Supervisor，并将T相关信息提交到Zookeeper集群上

3. Supervisor会去Zookeeper集群上认领自己的Task，通知自己的Worker进程进行Task的处理



1.5 Spark Streaming

- Spark Streaming可整合多种输入数据源，如Kafka、Flume、HDFS，甚至是普通的TCP套接字。经处理后的数据可存储至文件系统、数据库，或显示在仪表盘里



1.5 Spark Streaming

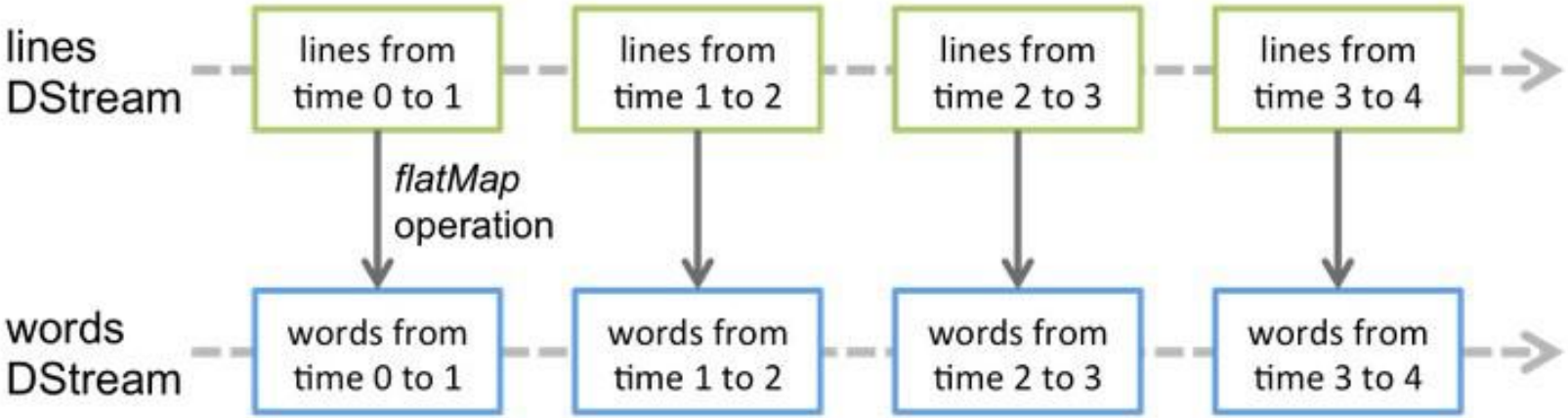
- Spark Streaming的基本原理是将实时输入数据流以时间片（秒级）为单位进行拆分，然后经Spark引擎以类似批处理的方式处理每个时间片数据



Spark Streaming执行流程

1.5 Spark Streaming

- Spark Streaming最主要的抽象是DStream（Discretized Stream，离散化数据流），表示连续不断的数据流。在内部实现上，Spark Streaming的输入数据按照时间片（如1秒）分成一段一段的DStream，每一段数据转换为Spark中的RDD，并且对DStream的操作都最终转变为对相应的RDD的操作



Dstream 操作示意图

1.5.1 Spark Streaming 与 Storm

- Spark Streaming无法实现毫秒级的流计算，而Storm可以实现毫秒级响应

- Spark Streaming构建在Spark上，因为Spark的低延迟执行引擎可以用于实时计算;
- 另一方面，相比于Storm，RDD数据集更容易做高效的容错处理



- Spark Streaming采用的小批量处理的方式使得它可以同时兼容批量和实时数据处理的逻辑和算法

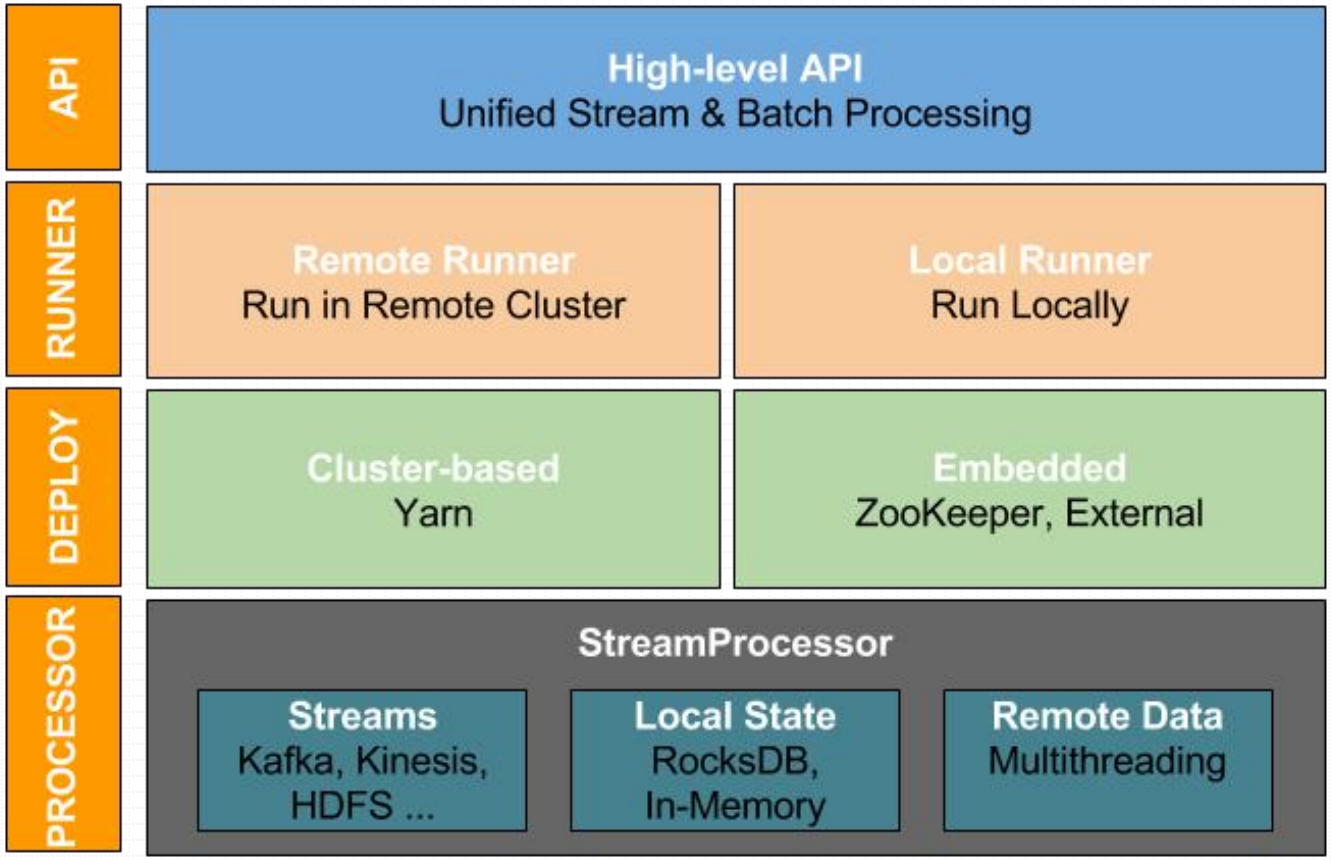
1.6 Samza

- Samza是一个高效流式数据处理框架，一个优秀的框架需要考虑多方面因素。例如生产者和消费者进程的管理，作业调度和容错处理，辅助工具和监控管理手段，更友好方便的用户接口等等，本质上说，Samza是在消息队列系统上的更高层的抽象，是一种应用流式处理框架在消息队列系统上的一种应用模式的实现

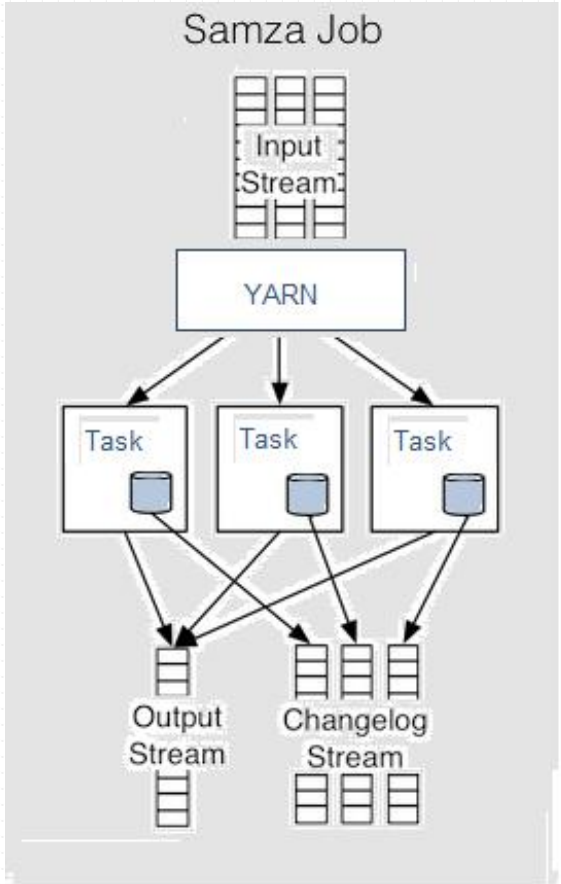


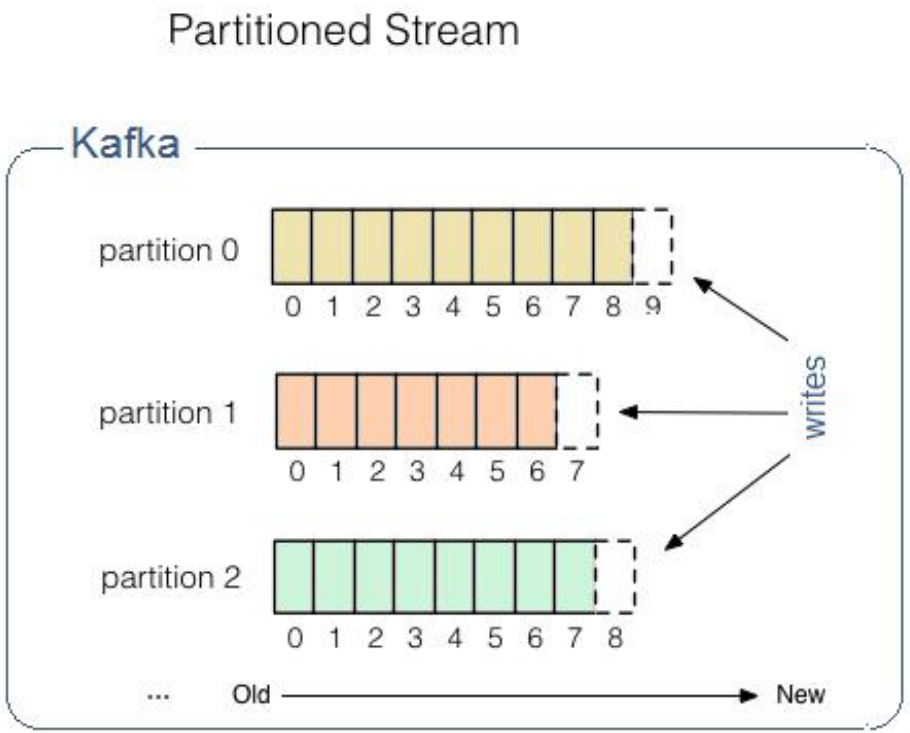
1.6.1 Smaza 架构

- Samza系统架构主要包括：流数据层（Kafka）、执行层（YARN）、处理层（Samza API）
- 流处理层和执行层都被设计成可插拔的，开发人员可以使用其他框架来替代YARN和Kafka



1.6.2 Smaza 基本概念





samza

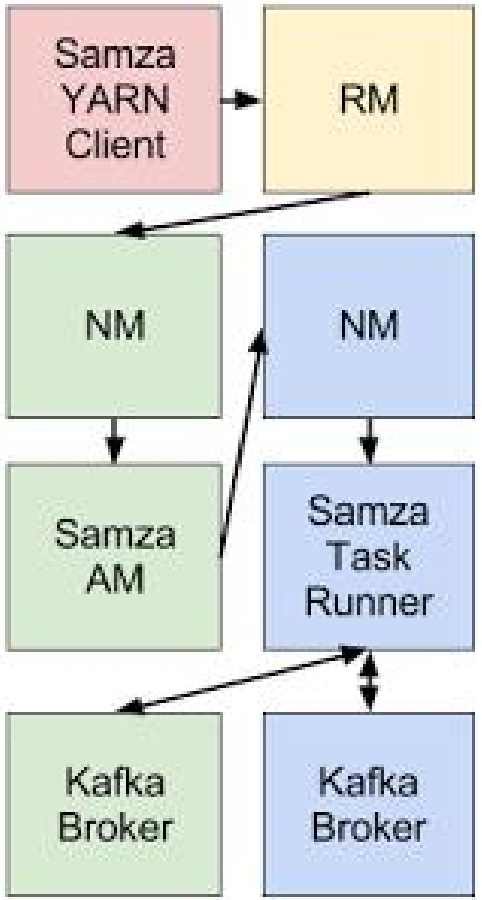
作业（Job）：是对一组输入流进行处理转化成输出流的程序。

分区（Partition）：Samza的流数据单位是一条条消息。Samza中的每个流都被分割成一个或多个分区，对于流里的每一个分区而言，都是一个有序的消息序列，后续到达的消息会根据一定规则被追加到其中一个分区里

任务(Task)：一个作业会被进一步分割成多个任务（Task）来执行，其中，每个任务负责处理作业中一个分区

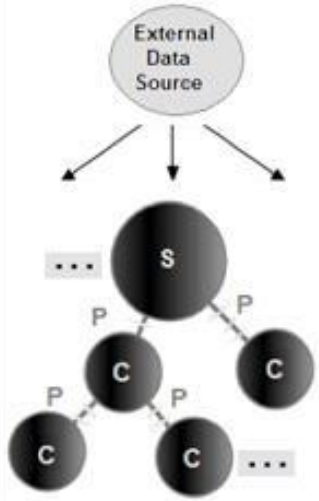
- 分区之间没有定义顺序，从而允许每一个任务独立执行
- YARN调度器负责把任务分发给各个机器，最终，一个工作中的多个任务会被分发到多个机器进行分布式并行处理

1.6.3 Smaza 处理流程



1. Samza客户端需要执行一个Samza作业时，它会向YARN的ResouceManager提交作业请求
2. ResouceManager通过与NodeManager沟通为该作业分配容器（包含了CPU、内存等资源）来运行Samza ApplicationMaster
3. Samza ApplicationMaster进一步向ResourceManager申请运行任务的容器
4. 获得容器后，Samza ApplicationMaster与容器所在的NodeManager沟通，启动该容器，并在其中运行Samza Task Runner
5. Samza Task Runner负责执行具体的Samza任务，完成流数据处理分析

1.6.4 Storm、SparkStreaming、Smaza 对比



	Storm	Spark	Samza
Stream Source (S)	Spouts	Receivers	Consumers
Stream Primitive (P)	Tuple	DStream	Message
Stream Computation (C)	Bolts	Transformations Window operations	Tasks

	Storm	Spark	Samza
Delivery Semantics	At Least Once Exactly-Once with Trident	Exactly Once Except in some failure scenarios	At Least Once
State Management	Stateless Roll your own or use Trident	Stateful Writes state to storage	Stateful Embedded key-value store
Latency	Sub-Second	Seconds Depending on batch size	Sub-Second
Language Support	Any JVM-languages, Ruby, Python Javascript, Perl.	Scala, Java, Python	Scala, Java JVM-languages only



流计算与图计算

CONTENTS

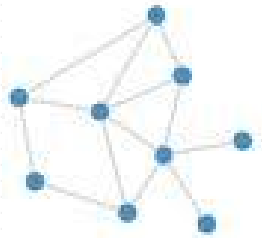
01 流计算
Streaming computation

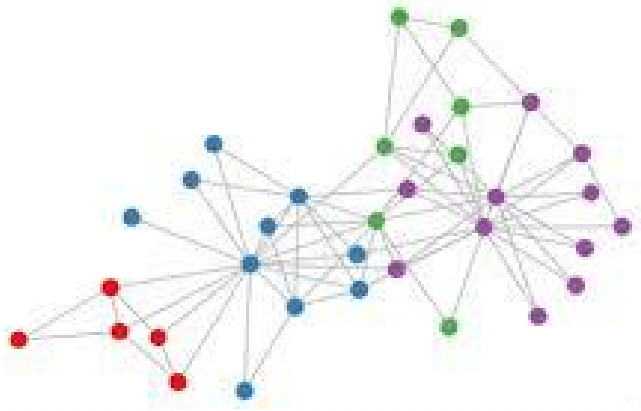
02 图计算
Graph computation

2.1 图计算概述

- 图（Graph）是一种重要的数据结构，它由节点V（或称为顶点，即个体），与边E（即个体之间的联系）构成，我们一般将图表示为G（V，E）。

- 图数据的典型例子有网页链接关系、社交网络、商品推荐等。对应互联网来说，可以把web网页看作顶点，页面之间的超链接关系作为边；对应社交网络来说，可以把用户看作顶点，用户之间建立的关系看作边。





2.1.1 传统图计算解决方案不足

传统图计算典型问题

- 常常表现出比较差的内存访问局部性
- 针对单个顶点的处理工作过少
- 计算过程中伴随着并行度的改变

大型图传统图计算不足

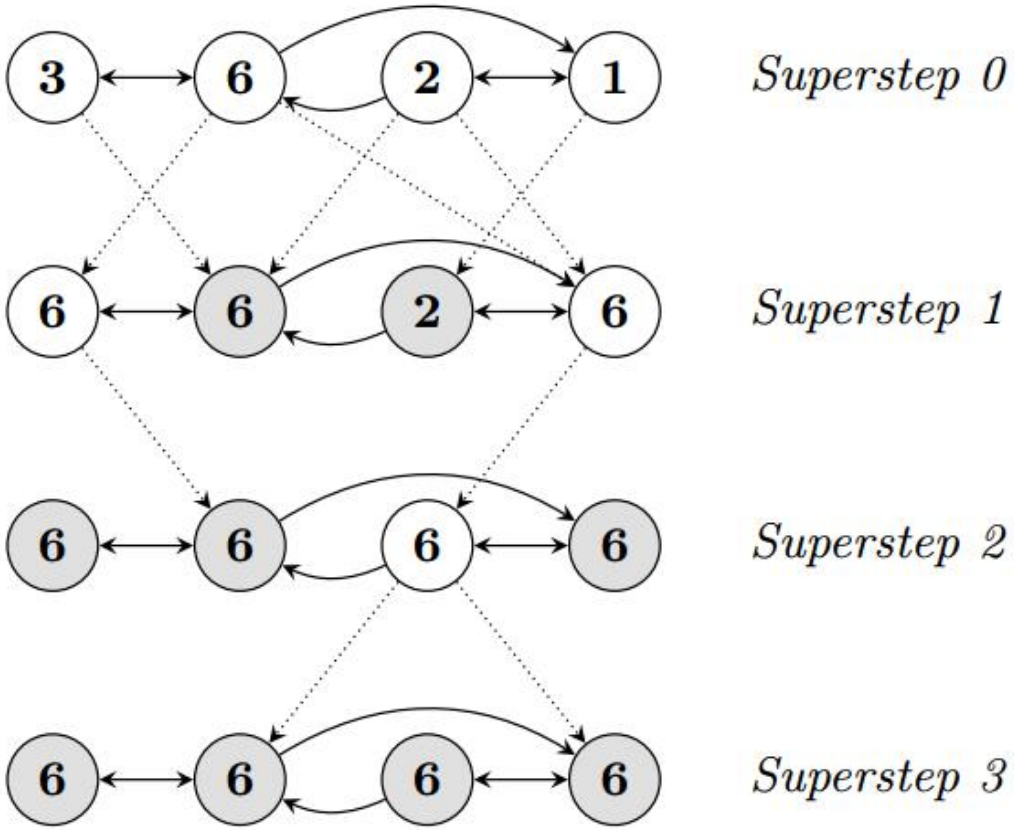
- **为特定图应用定制实现**：通用性不好
- **基于现有分布式计算平台**：性能和易用无法达到最优
  - 现有并行计算框架像MapReduce无法满足复杂的关联性计算
  - 有公司利用MapReduce进行社交用户推荐，对于5000万注册用户，50亿关系对利用10台机器的集群，需超过10个小时计算
- **使用单机图算法库**：BGL、LEAD、等，但是，在可以解决的问题的 规模方面具有很大的局限性
- **使用已有并行图计算系统**：Parallel BGL和CGM Graph，但对大规模分布式系统无较好支持

2.1.2 图计算通用软件

- 传统的图计算解决方案无法解决大型图的计算问题，因此，就需要设计能够用来解决这些问题的通用图计算软件

- 针对大型图的计算，目前通用的图计算软件主要包括两种：
  - 第一种主要是**基于遍历算法的、实时的图数据库**，如Neo4j、OrientDB、DEX和Infinite Graph
  - 第二种则是**以图顶点为中心的、基于消息传递批处理的并行引擎**，如GoldenOrb、Giraph、Pregel和Hama，这些图处理软件主要是基于BSP模型实现的并行图处理系统

2.1.2 图计算通用软件BSP



一次BSP(Bulk Synchronous Parallel Computing Model)计算过程包括一系列全局超步（所谓的超步就是计算中的一次迭代），每个超步主要有三个组件：

- **局部计算**：每个参与的**处理器**都有自身的计算任务，它们只读取存储在本地内存中的值，不同处理器的计算任务都是异步并且独立的
- **通讯**：处理器群相互交换数据，交换是由一方发起推送(put)和获取(get)操作
- **栅栏同步**(Barrier Synchronization)：当一个处理器遇到“路障” 会等到其他所有处理器完成它们的计算步骤；每一次同步也是一个超步的完成和下一个超步的开始

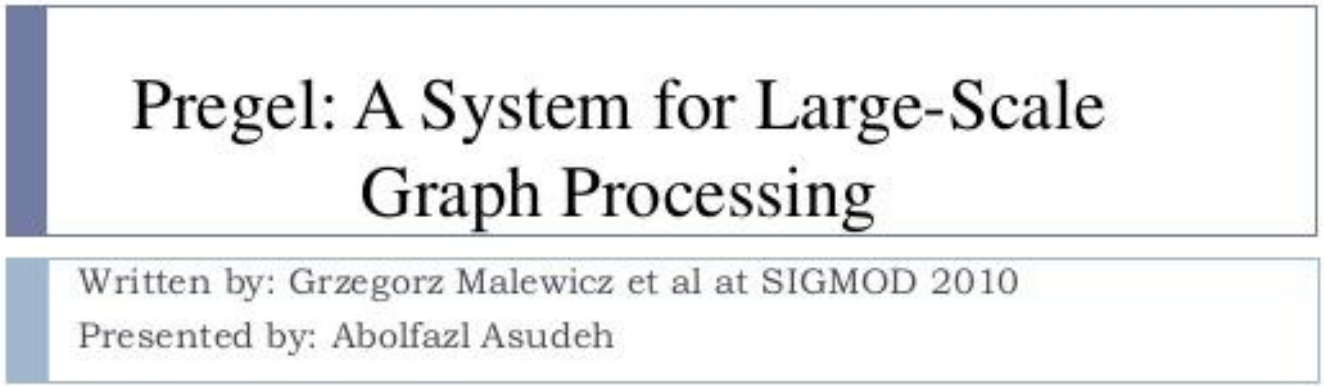
2.2 Pregel 简介

- 谷歌公司在2003年到2004年公布了GFS、MapReduce和BigTable，成为后来云计算和Hadoop项目的重要基石

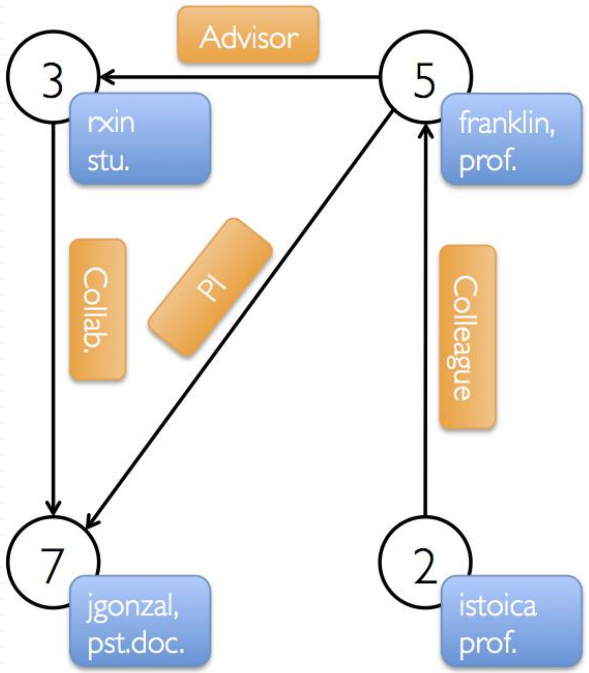
- 谷歌在后Hadoop时代的新“三驾马车”——Caffeine、Dremel和Pregel，再一次影响大数据技术的发展潮流

2.2 Pregel 简介

- Pregel是一种基于BSP模型实现的并行图处理系统
- 为了解决大型图的分布式计算问题，Pregel搭建了一套可扩展的、有容错机制的平台，该平台提供了一套非常灵活的API，可以描述各种各样的图计算
- Pregel作为分布式图计算的计算框架，主要用于图遍历、最短路径、PageRank计算等等



2.3 Pregel 图计算模型



Vertex Table

Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

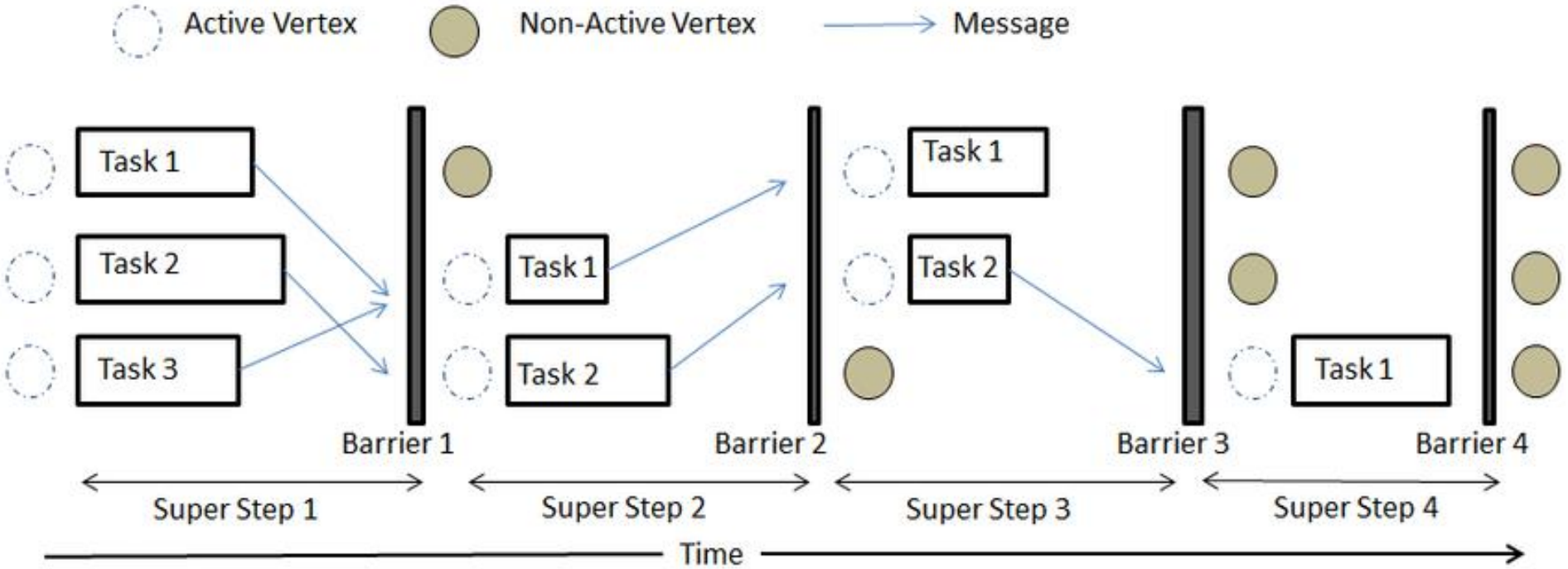
Edge Table

SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI

- Pregel计算模型以有向图作为输入
- 有向图的每个顶点都有一个String类型的顶点ID
- 每个顶点都有一个可修改的用户自定义值与之关联
- 每条有向边都和其源顶点关联，并记录了其目标顶点ID
- 边上有一个可修改的用户自定义值与之关联

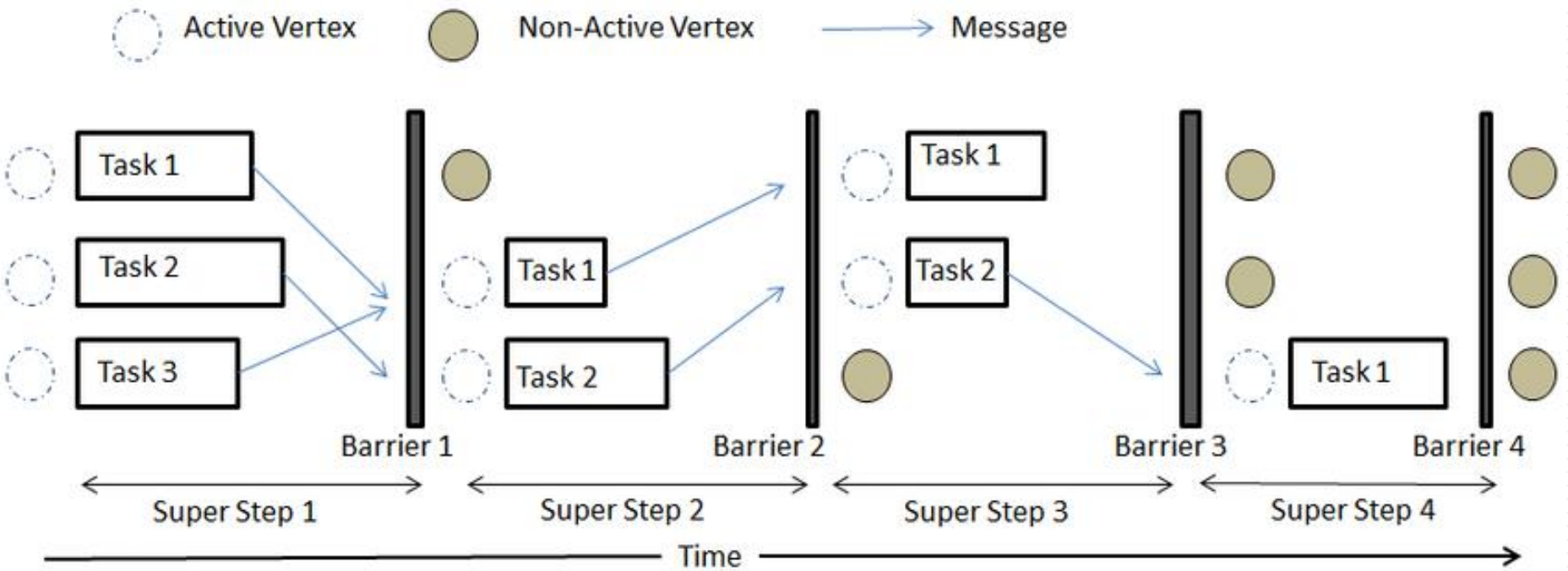
2.3 Pregel 图计算模型

- 在每个超步*S*中，图中的所有顶点都会并行执行相同的用户自定义函数
- 每个顶点可以接收前一个超步(*S*-1)中发送给它的消息，修改其自身及其出射边的状态，并发送消息给其他顶点，甚至是修改整个图的拓扑结构



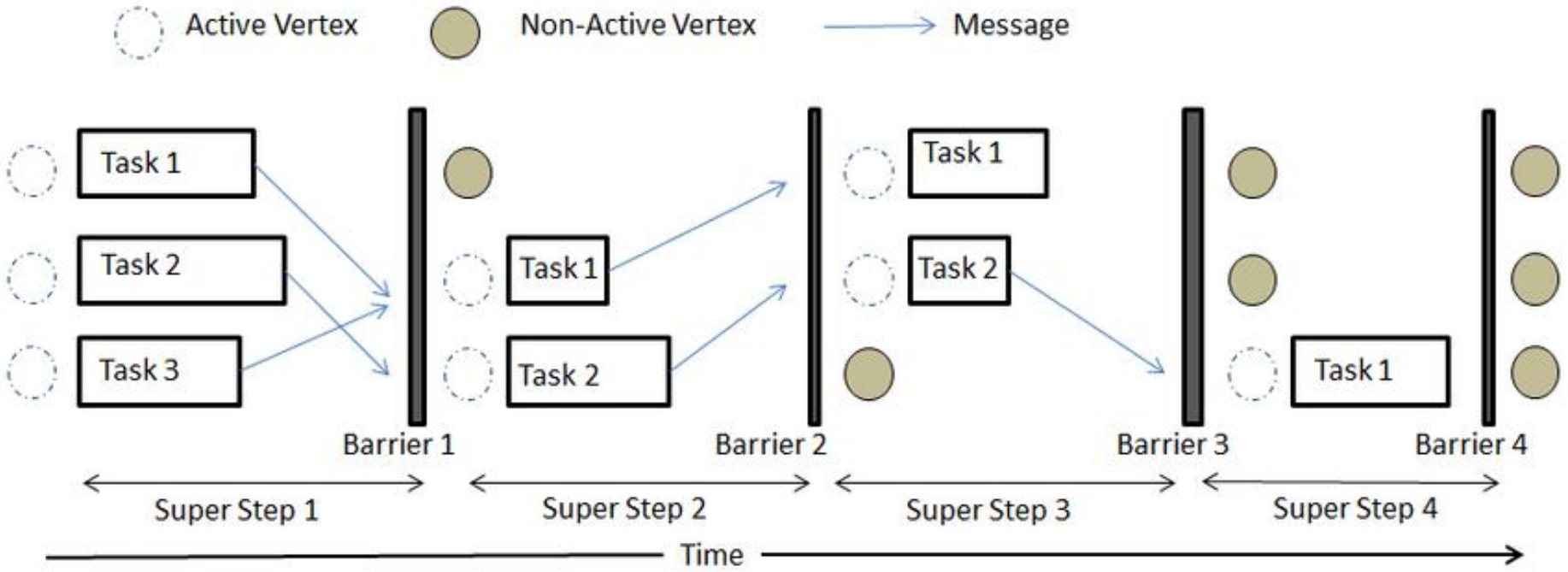
2.3 Pregel 图计算模型

- 在这种计算模式中，“边”并不是核心对象，边上不会运行相应计算，只有顶点才会执行用户自定义函数进行相应计算



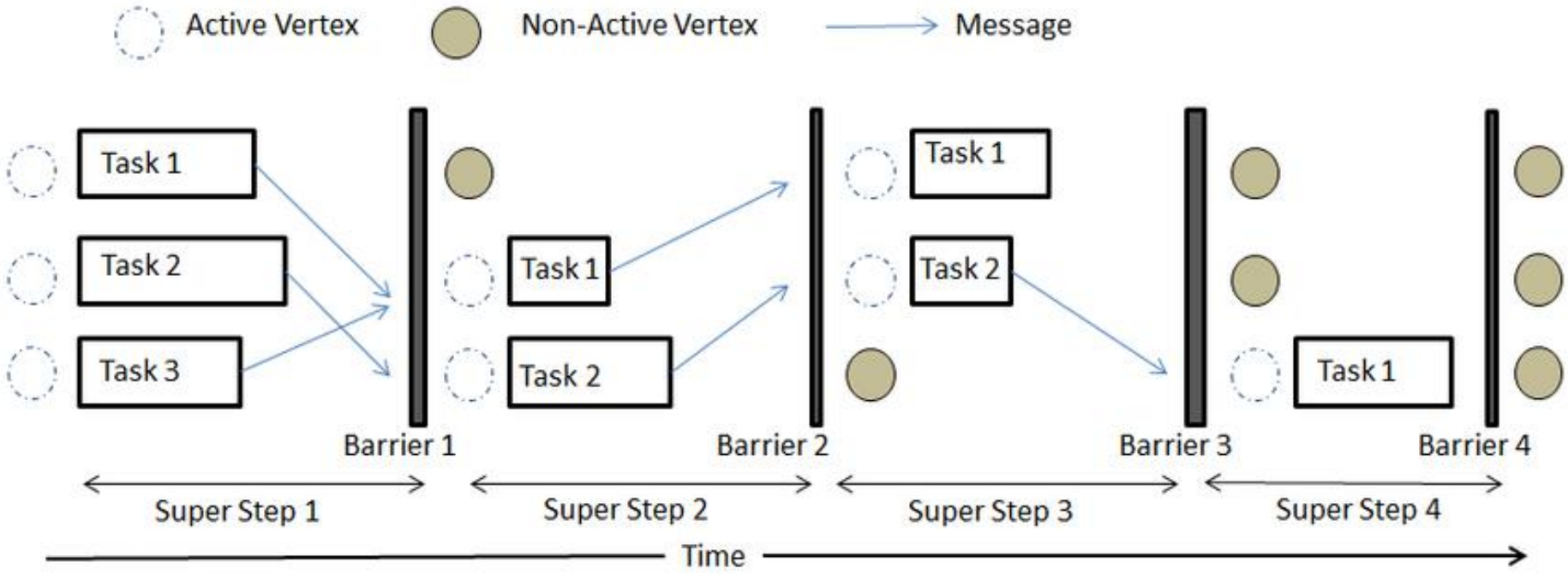
2.3.1 Pregel 图计算过程

1. Pregel的计算过程是由一系列被称为“超步”的迭代组成的 ，在每个超步中，每个顶点上面都会并行执行用户自定义的函数，该函数描述了一个顶点V在一个超步S中需要执行的操作



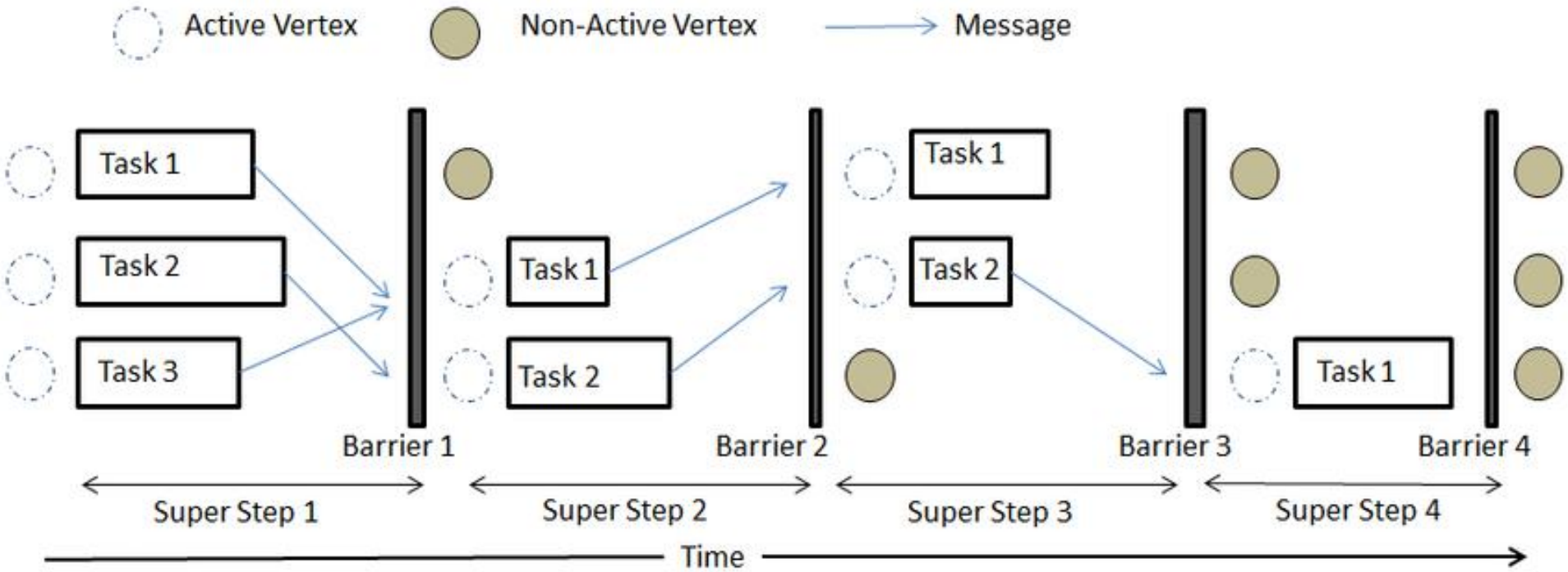
2.3.1 Pregel 图计算过程

2. 该函数可以读取前一个超步(S-1)中其他顶点发送给顶点V的消息，执行相应计算后，修改顶点V及其出射边的状态，然后沿着顶点V的出射边发送消息给其他顶点，而且，一个消息可能经过多条边的传递后被发送到任意已知ID的目标顶点上去



2.3.1 Pregel 图计算过程

3. 这些消息将会在下一个超步(S+1)中被目标顶点接收，然后象上述过程一样开始下一个超步(S+1)的迭代过程

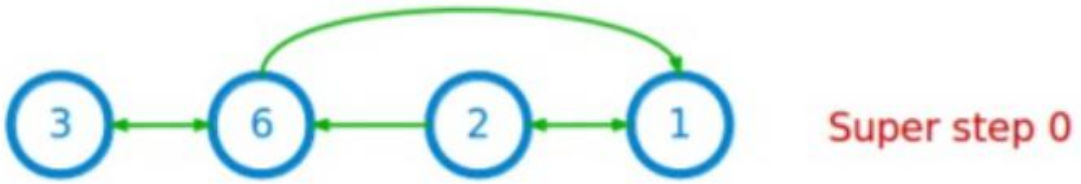


2.3.2 Pregel 求最大值计算实例（1/4）

```
i_val := val

for each message m
  if m > val then val := m

if i_val == val then
  vote_to_halt
else
  for each neighbor v
    send_message(v, val)
```

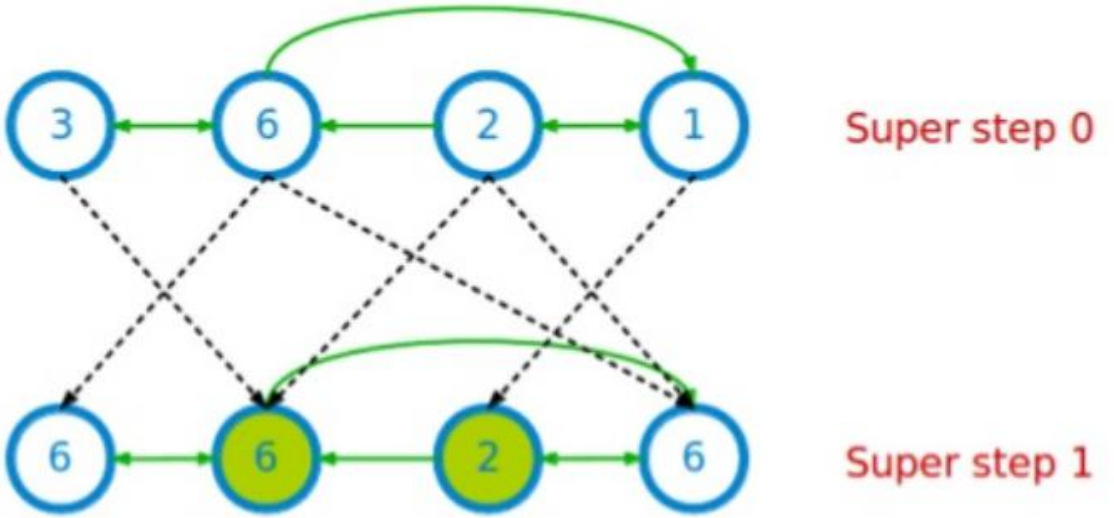


2.3.2 Pregel 求最大值计算实例（2/4）

```
i_val := val

for each message m
  if m > val then val := m

if i_val == val then
  vote_to_halt
else
  for each neighbor v
    send_message(v, val)
```

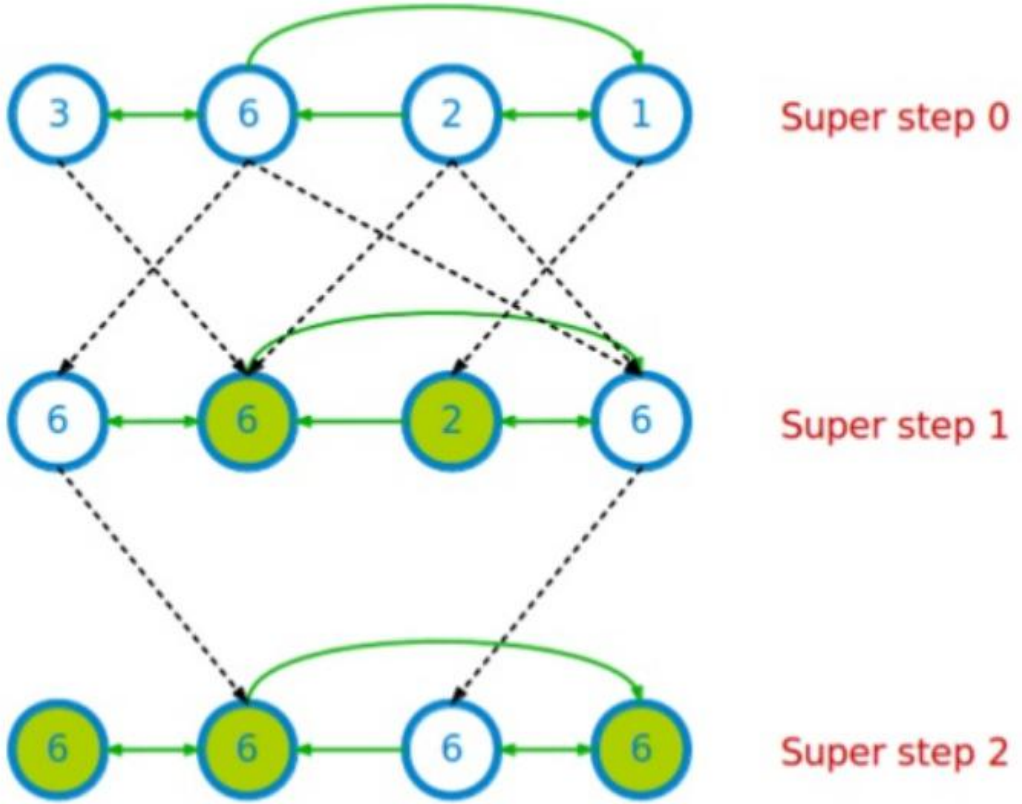


2.3.2 Pregel 求最大值计算实例（3/4）

```
i_val := val

for each message m
  if m > val then val := m

if i_val == val then
  vote_to_halt
else
  for each neighbor v
    send_message(v, val)
```

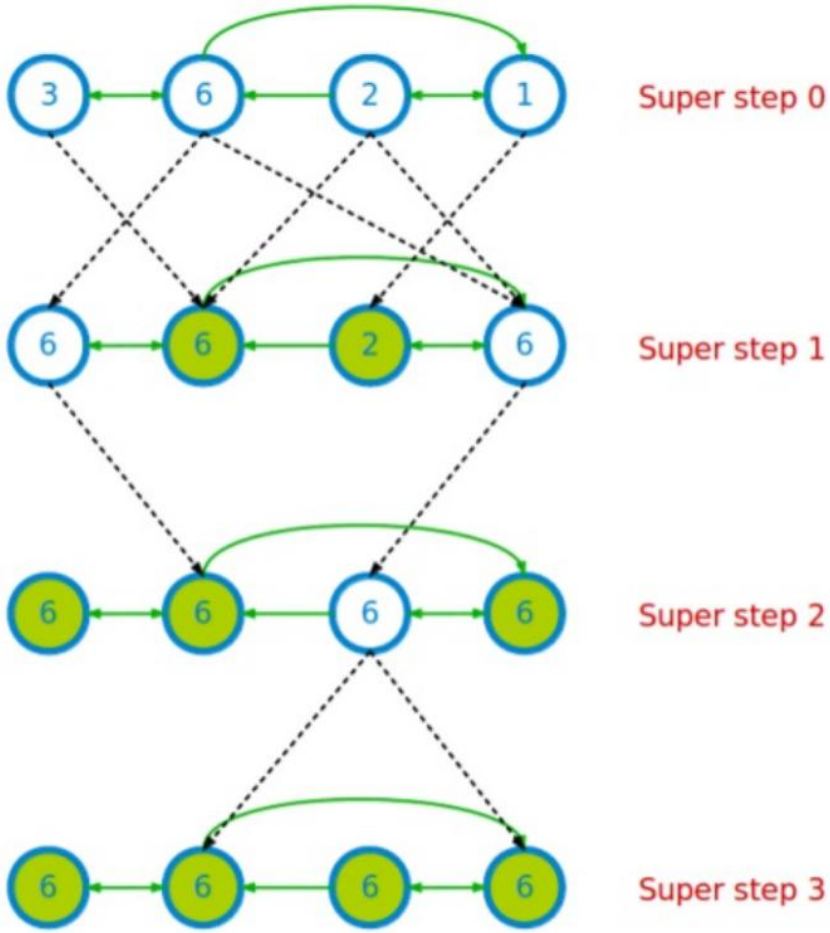


2.3.2 Pregel 求最大值计算实例（4/4）

```
i_val := val

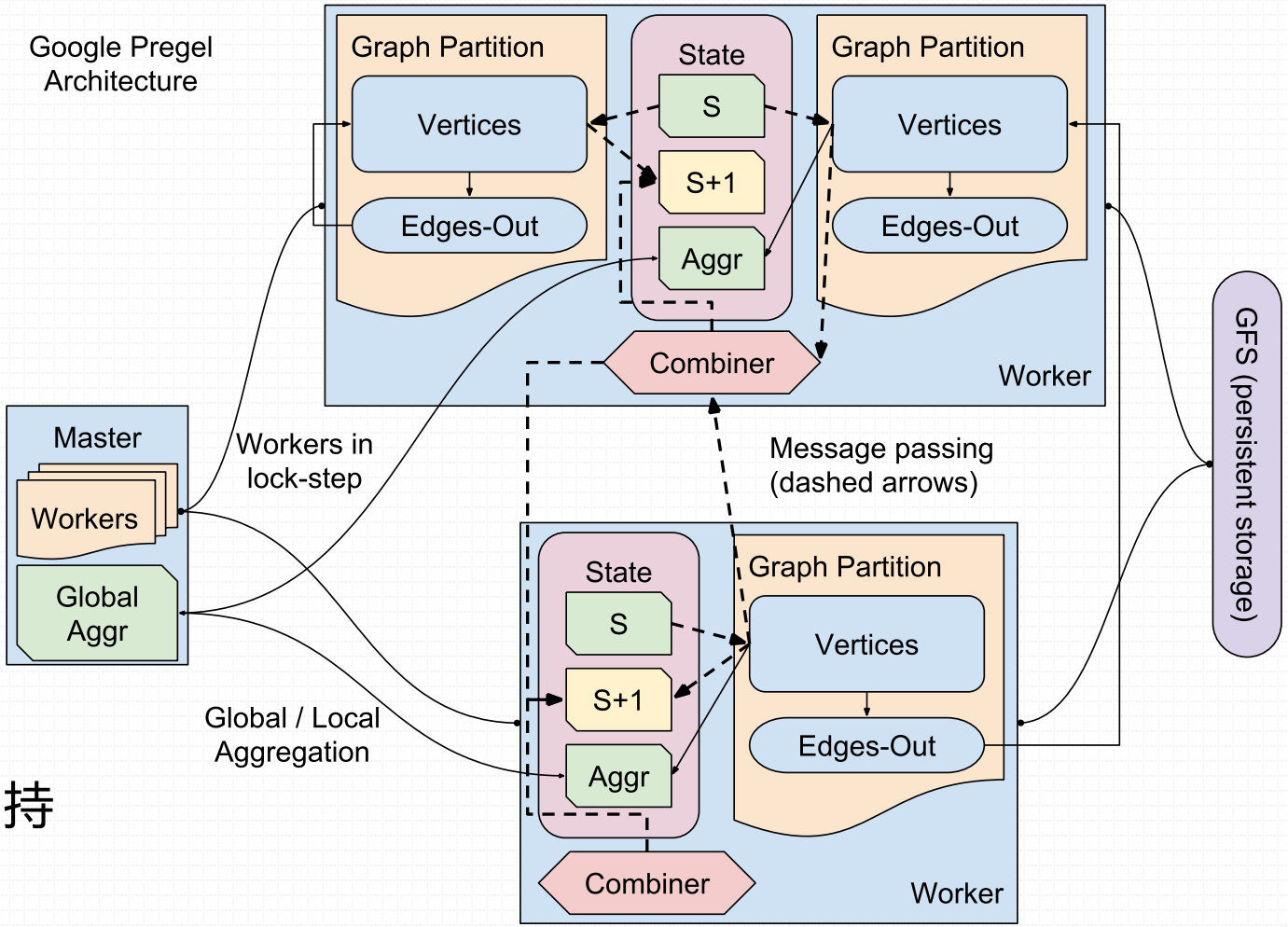
for each message m
  if m > val then val := m

if i_val == val then
  vote_to_halt
else
  for each neighbor v
    send_message(v, val)
```



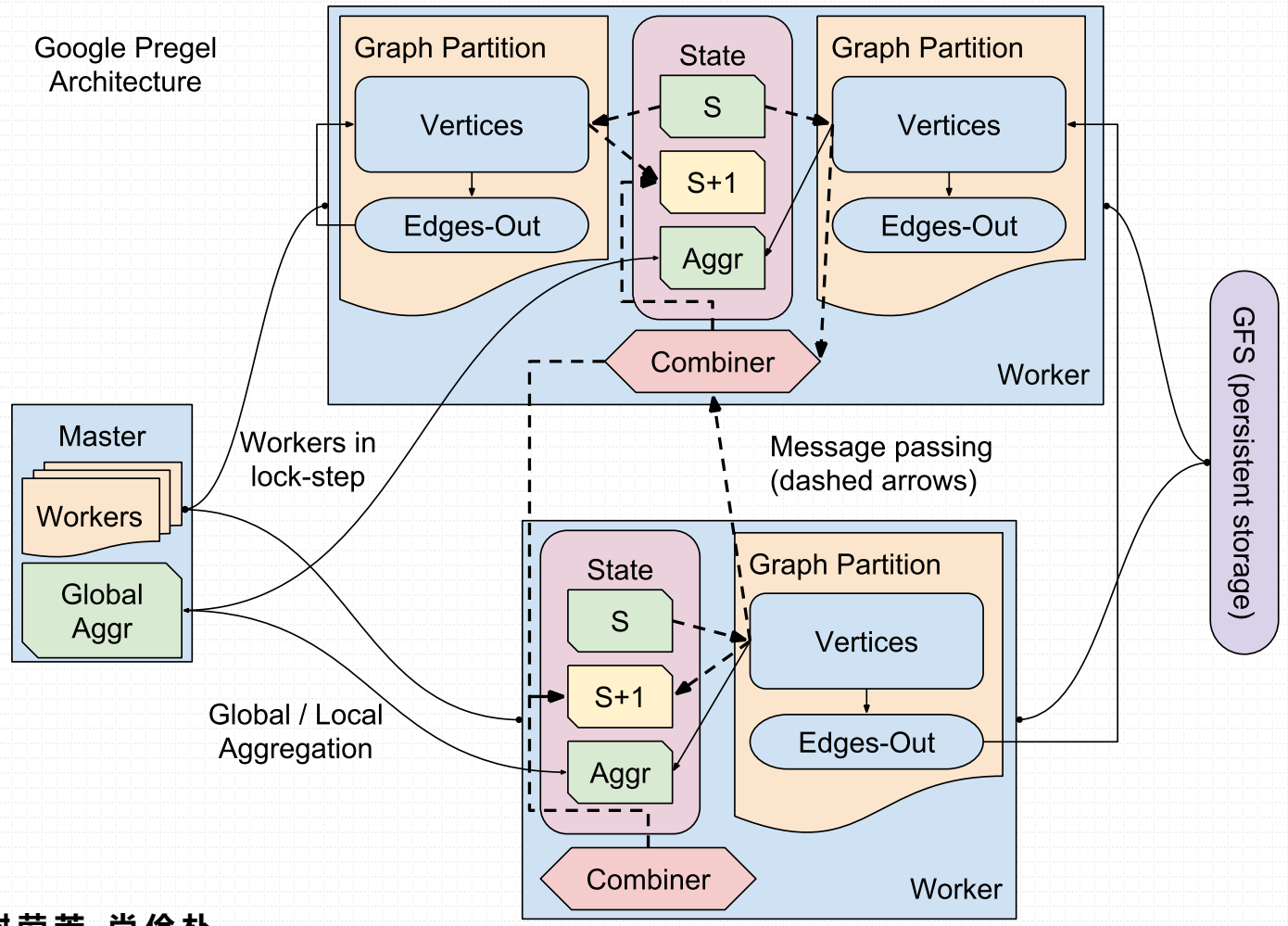
2.4 Pregel 体系结构

- Pregel架构采用Master-Worker模式
  - Master
    - 维护worker
    - 修复worker错误
    - 对任务进程提供Web-UI管理
  - Worker
    - 完成任务
    - 和其它worker关联
- 计算过程中数据均存储于集群内存
- 临时文件存储在节点的本地外存上，持久化数据存储在分布式文件系统中



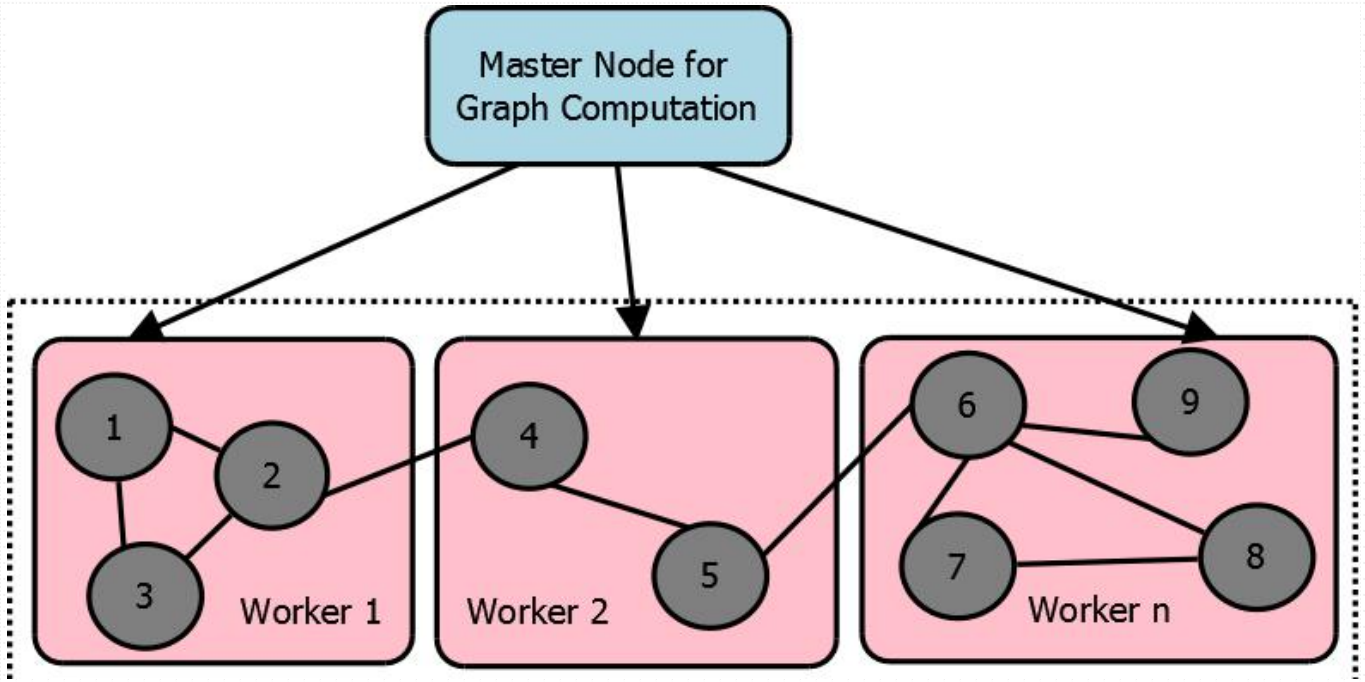
2.4 Pregel 体系结构

- 在Pregel中，一个大型图会被划分多个分区，每个分区都包含一部分顶点以及以其为起点边
- 一个顶点被分配到哪个分区上，由一个函数决定的，系统默认函数为hash(ID) mod *N*，其中，*N*为所有分区总数，ID是顶点的标识符
- 无论在哪台机器上，都可以简单根据顶点ID判断出该顶点属于哪个分区，即使该顶点不存在



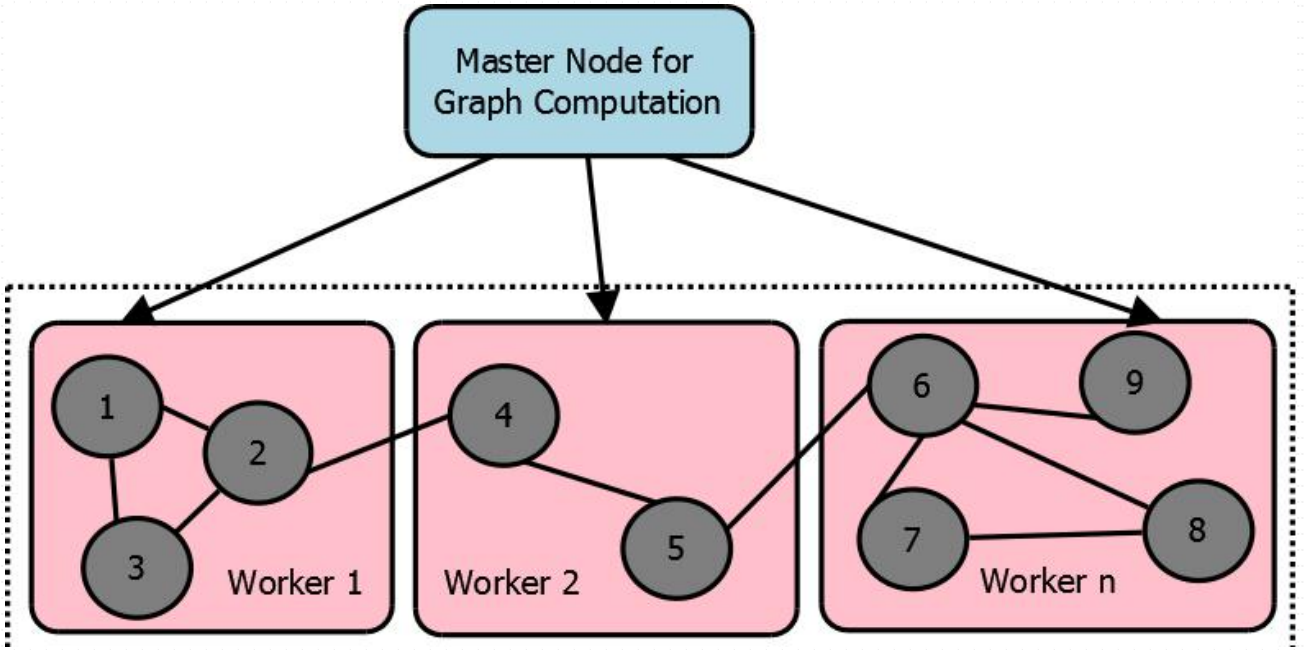
2.4.1 Pregel 执行过程(1/4)

1. 选择集群中的多台机器执行图计算任务，每台机器上运行用户程序的一个副本，其中一台机器会被选为Master，其他机器为Worker。Master只负责协调多个Worker执行任务，系统不会把图的任何分区分配给它。Worker借助于名称服务系统可以定位到Master的位置，并向Master发送自己的注册信息。



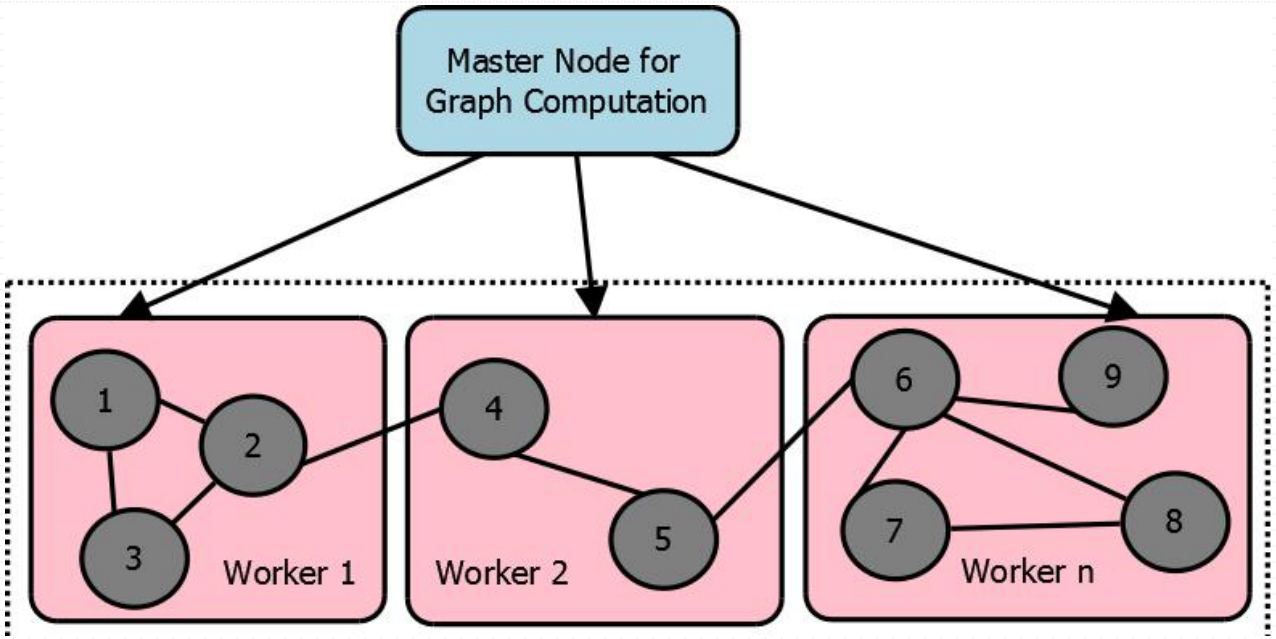
2.4.1 Pregel 执行过程(2/4)

2. Master把一个图分成多个分区，并把分区分配到多个Worker。一个Worker会领到一个或多个分区，每个Worker知道所有其他Worker所分配到的分区情况。每个Worker负责维护分配给自己的那些分区的状态(顶点及边的增删)，对分配给自己的分区中的顶点执行Compute()函数，向外发送消息，并管理接收到的消息。



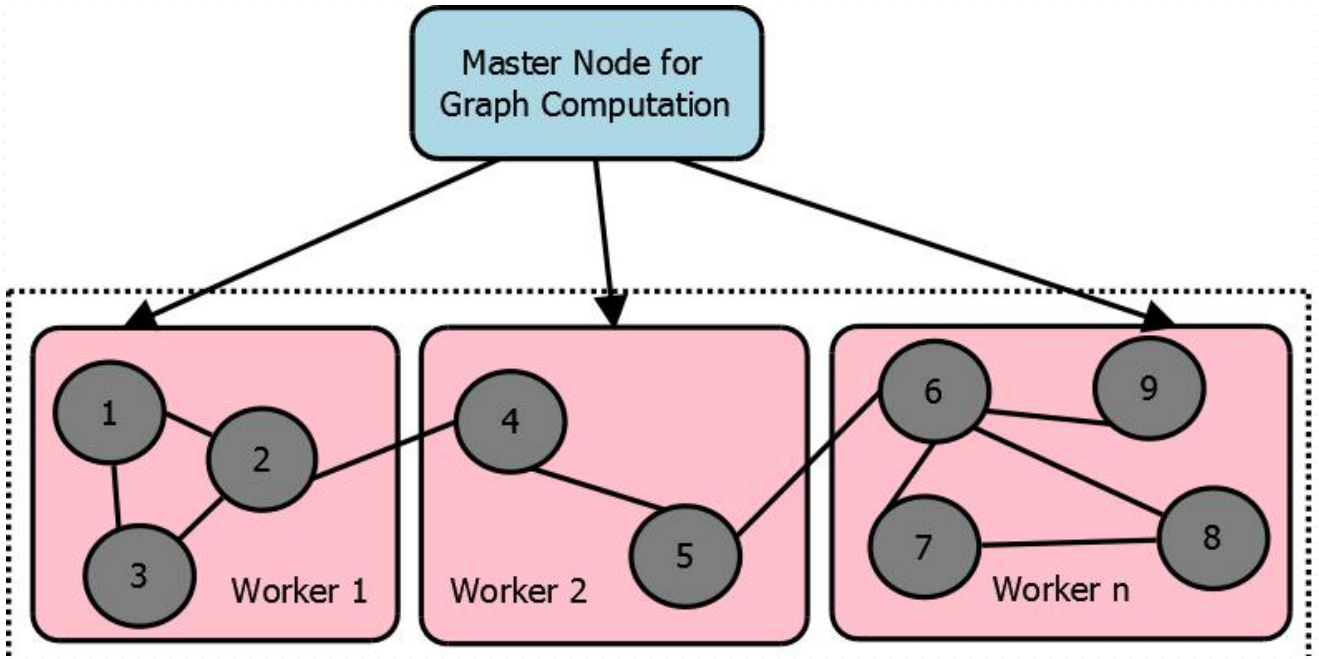
2.4.1 Pregel 执行过程(3/4)

3. Master会把用户输入划分成多个部分，通常是基于文件边界进行划分。划分后，每个部分都是一系列记录的集合，每条记录都包含一定数量的顶点和边。



2.4.1 Pregel 执行过程(3/4)

4. Master会为每个Worker分配用户输入的一部分。如果一个Worker从输入内容中加载到的顶点，刚好是自己所分配到的分区中的顶点，就会立即更新相应的数据结构。否则，该Worker会根据加载到的顶点的ID，把它发送到其所属的分区所在的Worker上。当所有的输入都被加载后，图中的所有顶点都会被标记为“活跃”状态。



2.4.2 Pregel 容错性

- Pregel采用检查点机制来实现容错。在每个超步的开始，Master会通知所有的Worker把自己管辖的分区的状态（包括顶点值、边值以及接收到的消息），写入到持久化存储设备

- Master会周期性地向每个Worker发送ping消息，Worker收到ping消息后会给Master发送反馈消息。如果Master在指定时间间隔内没有收到某个Worker的反馈消息，就会把该Worker标记为“失效”。同样地，如果一个Worker在指定的时间间隔内没有收到来自Master的ping消息，该Worker也会停止工作

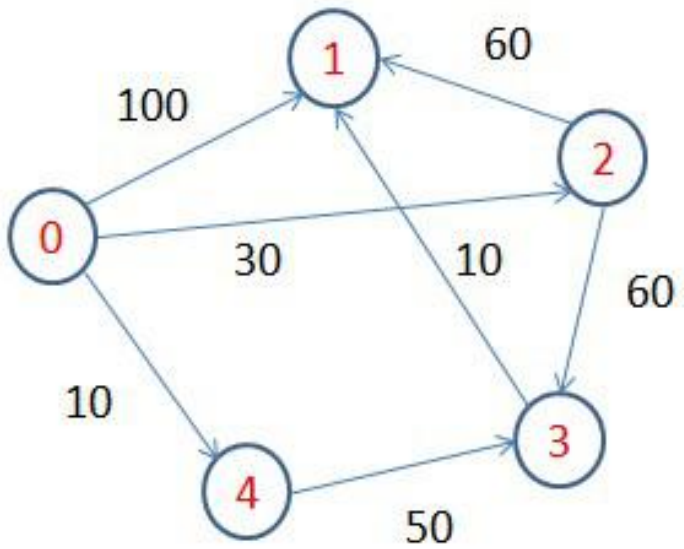
2.4.2 Pregel 容错性

- 每个Worker上都保存了一个或多个分区的状态信息，当一个Worker发生故障时，它所负责维护的分区的当前状态信息就会丢失。Master监测到一个Worker发生故障“失效”后，会把失效Worker所分配到的分区，重新分配到其他处于正常工作状态的Worker集合上，然后，所有这些分区会从最近的某超步S开始时写出的检查点中，重新加载状态信息

2.5 Pregel 应用实例--单源最短路径

Pregel 非常适合用来解决单源最短路径问题，实现代码如下：

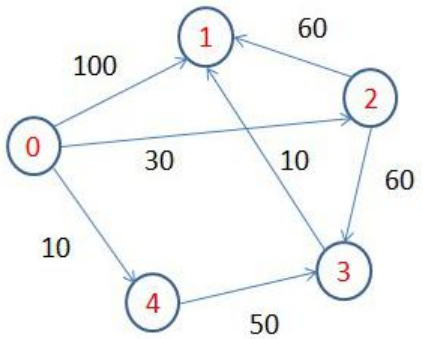
```
class ShortestPathVertex
        : public Vertex<int, int, int> {
         void Compute(MessageIterator* msgs) {
         int mindist = IsSource(vertex_id()) ? 0 : INF;
         for (; !msgs->Done(); msgs->Next())
                mindist = min(mindist, msgs->Value());
         if (mindist < GetValue()) {
                 *MutableValue() = mindist;
                OutEdgeIterator iter =  GetOutEdgeIterator();
                for (; !iter.Done(); iter.Next())
                SendMessageTo(iter.Target(),
                mindist + iter.GetValue());
        }
        VoteToHalt();
        }
};
```



Dijkstra算法解决单源最短路径问题

2.5 Pregel 应用实例（1/2）

```
1  class ShortestPathVertex
2          : public Vertex<int, int, int> {
3          void Compute(MessageIterator* msgs) {
4          int mindist = IsSource(vertex_id()) ? 0 : INF;
5          for (; !msgs->Done(); msgs->Next())
6                  mindist = min(mindist, msgs->Value());
7          if (mindist < GetValue()) {
8                  *MutableValue() = mindist;
9                  OutEdgeIterator iter =  GetOutEdgeIterator();
10                 for (; !iter.Done(); iter.Next())
11                 SendMessageTo(iter.Target(),
12                 mindist + iter.GetValue());
13         }
14         VoteToHalt();
15         }
16 };
```



超 步 0

表1 超步0开始时的顶点值

	0	1	2	3	4
0	INF	INF	INF	INF	INF

表2 超步0结束时的顶点值

	0	1	2	3	4
0	0	INF	INF	INF	INF

表3 顶点0向其他顶点发送消息

	1	2	3	4
0	100	30	无	10

2.5 Pregel 应用实例（2/2）

I. **顶点0**：没有收到消息，依然非活跃

II. **顶点1**：收到消息100（唯一消息），被显式唤醒、计算，mindist变为100，小于顶点值INF，顶点值修改为100，没有出射边，不需要发送消息，最后变为非活跃

III. **顶点2**：收到消息30，被显式唤醒，执行计算，mindist变为30，小于顶点值INF，顶点值修改为30，有两条出射边，向顶点3发送消息90（即：30+60），向顶点1发送消息90（即：30+60），最后变为非活跃

表4 上步（超步0）中发出的消息

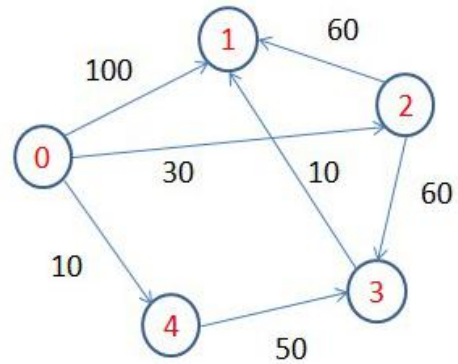
	1	2	3	4
0	100	30	无	10

表5 超步1开始时的顶点值

	0	1	2	3	4
0	0	INF	INF	INF	INF

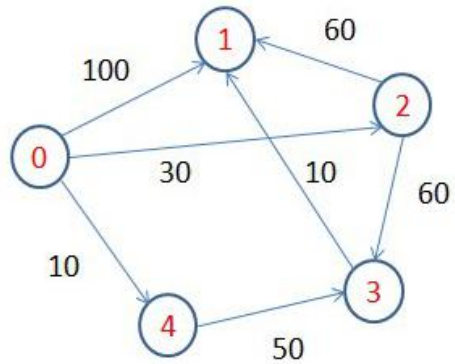
表6 超步1结束时的顶点值

	0	1	2	3	4
0	0	100	30	INF	10



超 步 1

2.5 Pregel 应用实例（2/2）



超 步 1

IV. 顶点3：没有收到消息，依然非活跃

V. 顶点4：收到消息10，被显式唤醒，执行计算， mindist变为10，小于顶点值INF，顶点值修改为10，向顶点3发送消息60（即：10+50），最后变为非活跃

表4 上步（超步0）中发出的消息

	1	2	3	4
0	100	30	无	10

表5 超步1开始时的顶点值

	0	1	2	3	4
0	0	INF	INF	INF	INF

表6 超步1结束时的顶点值

	0	1	2	3	4
0	0	100	30	INF	10

2.6 PageRank算法



2.6 PageRank算法

- PageRank是一个函数，它为网络中每个网页赋一个权值。通过该权值来判断该网页的重要性

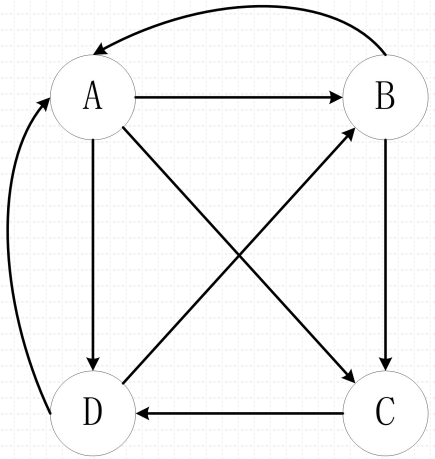
- 该权值分配的方法并不是固定的，对PageRank算法的一些简单变形都会改变网页的相对PageRank值。PageRank作为谷歌的网页链接排名算法，基本公式如下：

$$PR(Z) = \frac{(1-DF)}{N} + DF\sum_{i=1}^{N}\frac{PR(i)}{Nb(i)}$$

2.6 PageRank算法

- 对于任意一个网页链接，其PR值为链入到该链接的源链接的PR值对该链接的贡献和，其中，N表示该网络中所有网页的数量，N_i为第i个源链接的链出度，PR_i表示第i个源链接的PR值

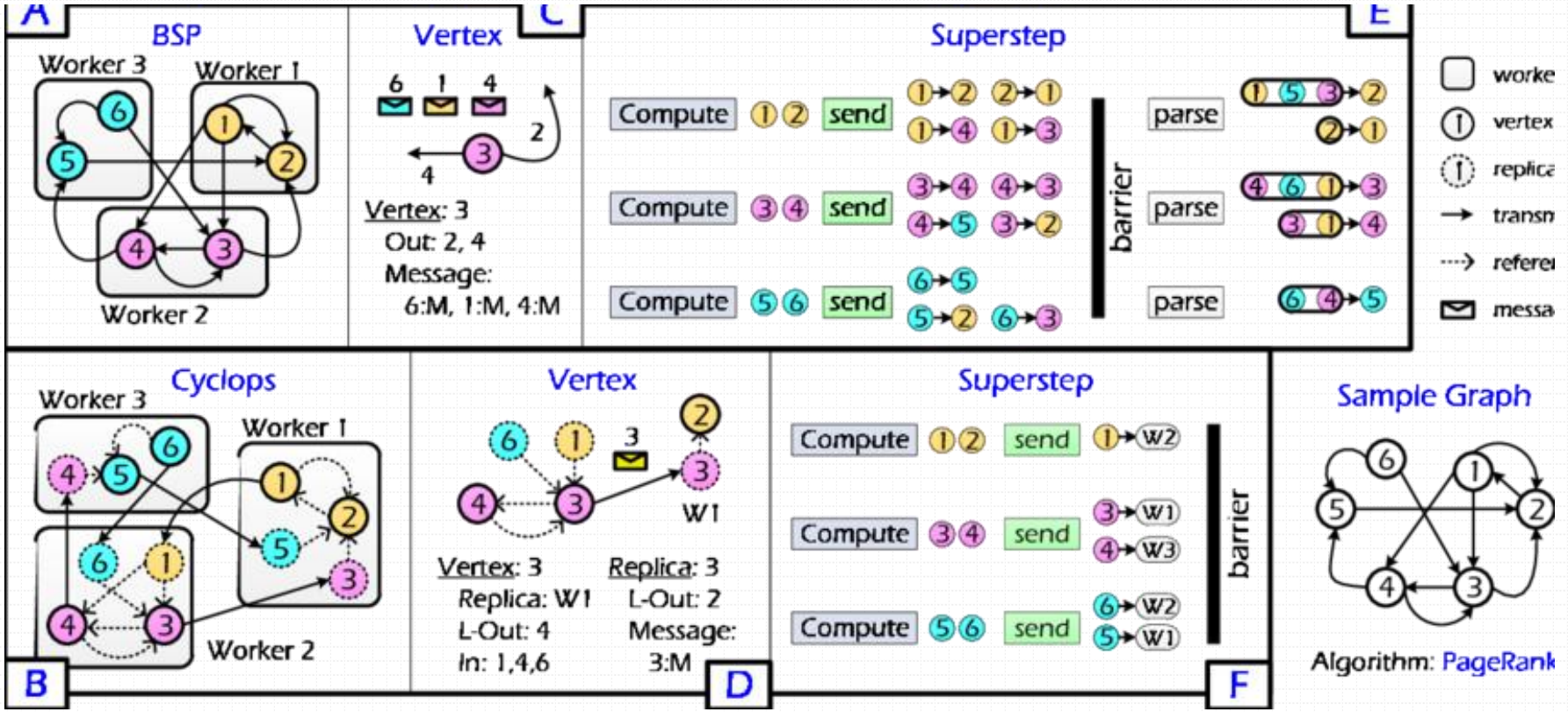
- 网络链接之间的关系可以用一个连通图来表示，下图就是四个网页（A,B,C,D）互相链入链出组成的连通图，从中可以看出，网页A中包含指向网页B、C和D的外链，网页B和D是网页A的源链接



重庆大学 CHONGQING UNIVERSITY

2.6.1 PageRank在Pregel中实现（1/3）

- 图中每个顶点对应一个计算单元，每个包含三个变量：
  - ➢**顶点值（Vertex value）**：顶点对应的PR值
  - ➢**出射边（Out edge）**：表示一条边，可不取值
  - ➢**消息（Message）**：传递的消息
- 每个计算单元包含一个成员函数Compute()



2.6.1 PageRank在Pregel中实现（2/3）

Superstep 0: Value of each vertex is **1/NumVertices()**

```
virtual void Compute(MessageIterator* msgs) {
        if (superstep() >= 1) {
                double sum = 0;
                for (; !msgs->done(); msgs->Next())
                        sum += msgs->Value();
                *MutableValue() = 0.15 + 0.85 * sum;
        }

        if (supersteps() < 30) {
                const int64 n = GetOutEdgeIterator().size();
                SendMessageToAllNeighbors(GetValue() / n);
        } else {
                VoteToHalt();
```

I. PageRankVertex继承自Vertex类，顶点值类型是double，用来保存PageRank中间值，消息类型也是double，用来传输PageRank值，边的value类型是void，因为不需要存储任何信息

II. 这里假设在第0个超步时，图中各顶点值被初始化为1/NumVertices()，其中，NumVertices()表示顶点数目

2.6.1 PageRank在Pregel中实现（2/3）

Superstep 0: Value of each vertex is **1/NumVertices()**

```
virtual void Compute(MessageIterator* msgs) {
      if (superstep() >= 1) {
            double sum = 0;
            for (; !msgs->done(); msgs->Next())
                  sum += msgs->Value();
            *MutableValue() = 0.15 + 0.85 * sum;
      }

      if (supersteps() < 30) {
            const int64 n = GetOutEdgeIterator().size();
            SendMessageToAllNeighbors(GetValue() / n);
      } else {
            VoteToHalt();
```

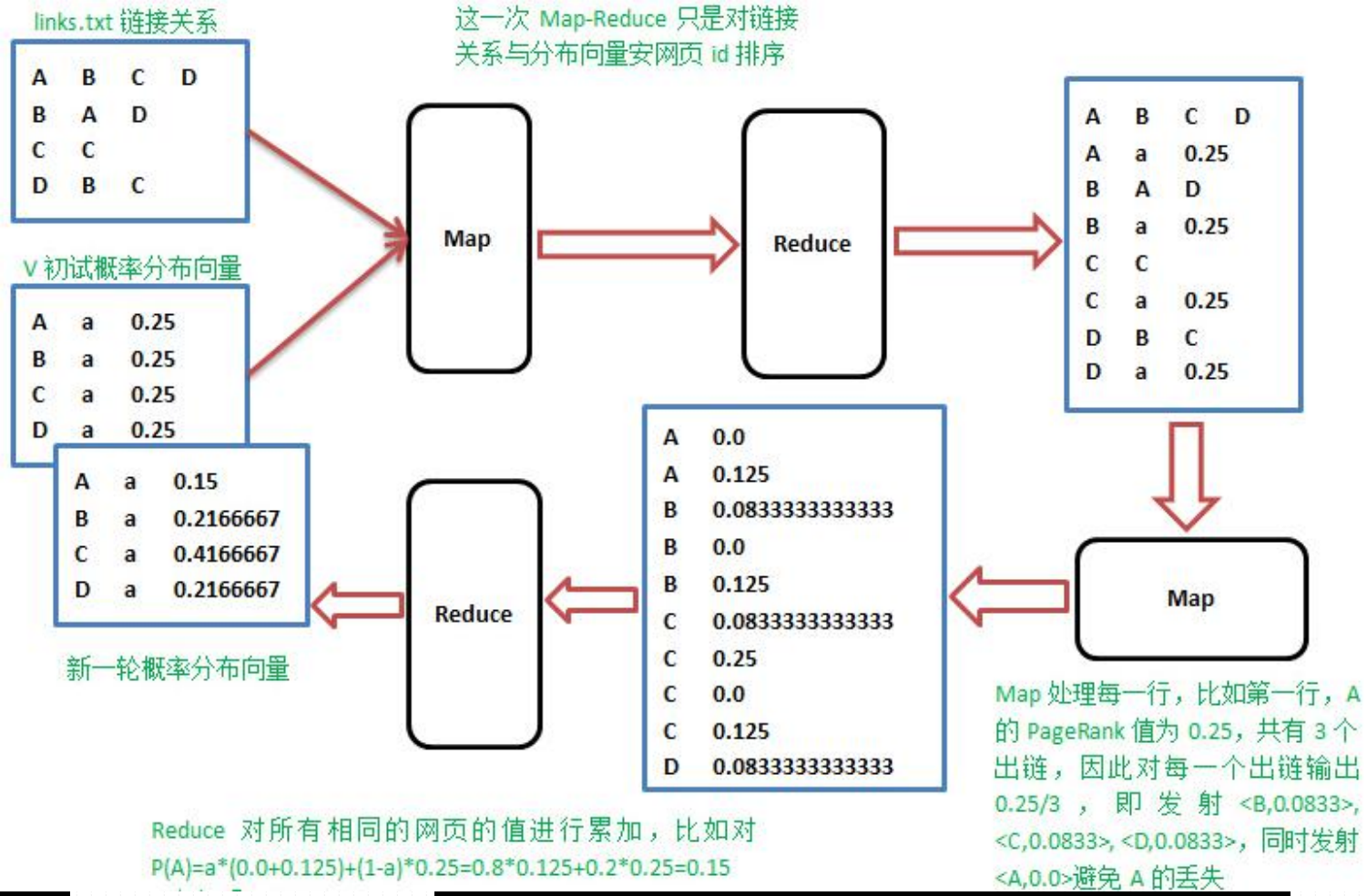
III. 在前30个超步中，每个顶点都会沿着它的出射边，发送它的PageRank值除以出射边数目以后的结果值。从第1个超步开始，每个顶点会将到达的消息中的值加到sum值中，同时将它的PageRank值设为0.15/NumVertices()+0.85*sum

IV. 到了第30个超步后，就没有需要发送的消息了，同时所有的顶点停止计算，得到最终结果

2.6.2 PageRank在MapReduce中实现（1/2）

- MapReduce也是谷歌公司提出的一种计算模型，它是为全量计算而设计
- 采用MapReduce实现PageRank的计算过程包括三个阶段：
  - **第一阶段：解析网页**
  - **第二阶段：PageRank分配**
  - **第三阶段：收敛阶段**

✓ 右图是一次MapReduce过程，不断迭代后PR值会趋于稳定收敛

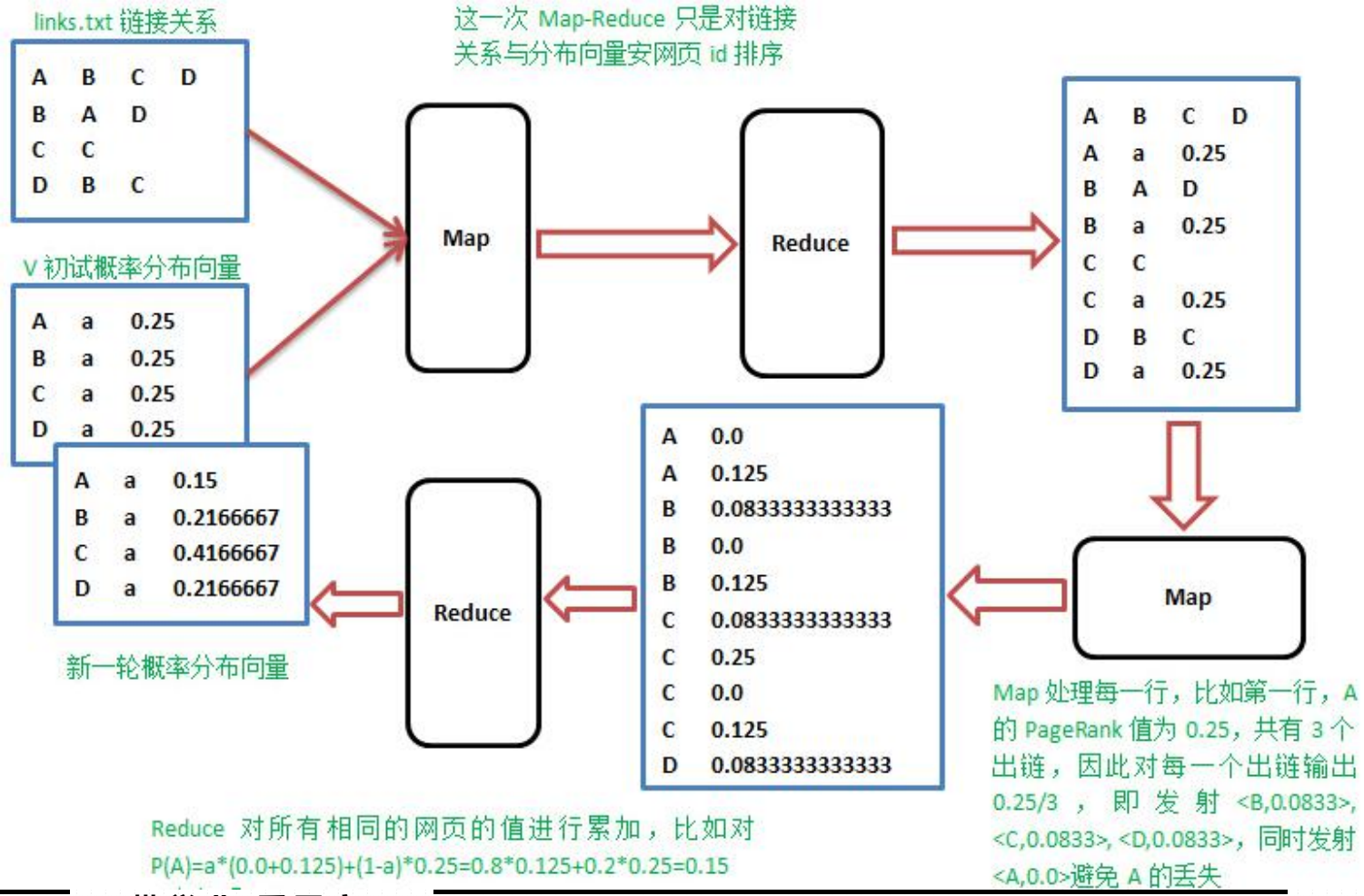


重庆大学 CHONGQING UNIVERSITY

2.6.2 PageRank在MapReduce中实现（2/2）

- ✓ 进行40次左右迭代后，pagerank值已经基本趋于稳定，并与第四步的分数表示一致。

```
38 0.101351351562 0.128378378686
0.641891891065 0.128378378686
39 0.101351351474 0.128378378558
0.64189189141 0.128378378558
40 0.101351351423 0.128378378483
0.641891891611 0.128378378483
41 0.101351351393 0.128378378439
0.641891891728 0.128378378439
```



2.6.3 PageRank在Pregel与MapReduce中对比

■ PageRank算法在Pregel和MapReduce区别

➢ Pregel将PageRank处理对象看成是连通图，而MapReduce则将其看成是键值对

➢ Pregel将计算细化到顶点，同时在顶点内控制循环迭代次数，而MapReduce则将计算批量化处理，按任务进行循环迭代控制

➢ 图算法如果用MapReduce实现，需要一系列的MapReduce的调用。从一个阶段到下一个阶段，它需要传递整个图的状态，会产生大量不必要的序列化和反序列化开销。而Pregel使用超步简化了这个过程



流计算与图计算

BIG DATA

Thank You!